UCS3461 Foundations of Artificial Intelligence

INFERENCE IN PL

Semester 4, B.E. Computer Science and Engineering
29 January 2026

Sakaya Milton R
SSN College of Engineering

Model Checking in PL

Inference by Model Checking

- Atoms

  P_{xy}: There is a pit in (x, y).
  W_{xy}: There is a wumpus in (x, y), dead or alive.
  B_{xy}: Breeze perceived in (x, y).
  S_{xy}: Stench perceived in (x, y).

- Knowledge Base K, Goal $\neg P_{12}$

$$R_1 : \neg P_{11}$$

$$R_2 : B_{11} \leftrightarrow P_{12} \vee P_{21}$$

$$R_3 : B_{21} \leftrightarrow P_{11} \vee P_{22} \vee P_{31}$$

$$R_4 : \neg B_{11}$$

$$R_5 : B_{21}$$

- Decide whether $K \models G$
- Decide whether $K \models \neg P_{12}$

Model Checking Procedure

	B_{11}	B_{21}	P_{11}	P_{12}	P_{21}	P_{22}	P_{31}	R_1	R_2	R_3	R_4	R_5	KB
1	F	F	F	F	F	F	F	F	F	F	F	F	F
2	F	F	F	F	F	F	T	T	T	F	T	F	F
	⋮												
	F	T	F	F	F	F	F	T	T	F	T	T	F
	F	T	F	F	F	F	T	T	T	T	T	T	T
	F	T	F	F	F	T	F	T	T	T	T	T	T
	F	T	F	F	F	T	T	T	T	T	T	T	T
	F	T	F	F	T	F	F	T	F	F	T	T	F
	⋮												
128	T	T	T	T	T	T	T	F	T	T	F	T	F

- $2^7 = 128$ rows (assignments)
- 3 rows, R_1 through R_5 are true $\equiv$ KB is true
- In those 3 rows, $\neg P_{12}$ is true
- $KB \models \neg P_{12}$
- $KB \models P_{22}$?

Model Checking Algorithm

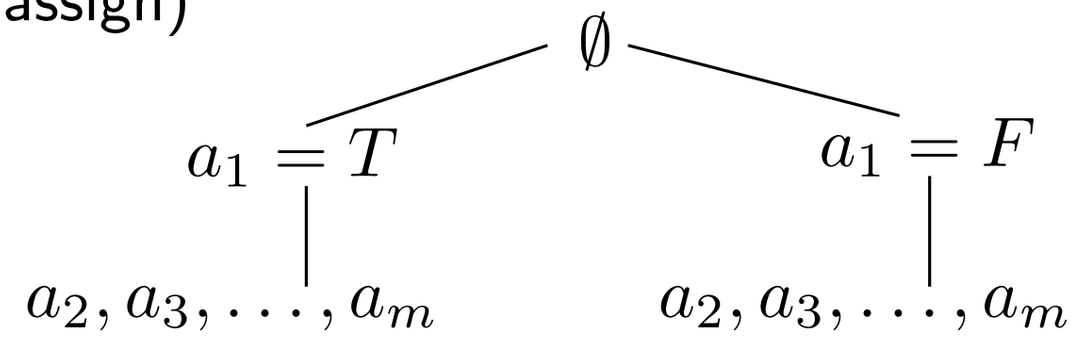
Algorithm: Entails_TT (KB, G, atoms, assign)

Input:

KB: set of clauses

G: goal clause

Output: Value of $KB \rightarrow G$



if atoms $=$ [ ] **then**

  return Evaluate($KB \rightarrow G$, assign)

a := First(atoms)

rest := Rest(atoms)

val_pos := Entails_TT(KB, G, rest , assign $\cup \{a = T\}$)

val_neg := Entails_TT(KB, G, rest, assign $\cup \{a = F\}$)

return val_pos $\wedge$ val_neg

Analysis of Model Checking Algorithm

Algorithm: Entails_TT_Wrapper(KB, G)

Input:

KB: knowledge base in PL
G: query in PL

Output: $KB \rightarrow G$

atoms := list of atoms in KB and G
return Entails_TT(KB, G, atoms, $\{\}$)

- Sound, complete (finitely many assignments)
- Recursive backtracking search: A has n atoms – 2^n assignments – $O(2^n)$ time-complexity
- $O(n)$ space-complexity
- PL entailment is Co-NP-complete (i.e., probably no easier than NP-complete)
- Every known inference algorithm for PL has a worst-case complexity, exponential in the size of the input.

Theorem Proving (Inference) in PL

Proofs

$$\frac{a \wedge b}{a}$$

$$a \wedge b \vdash a$$

Inference Rules

$\dfrac{a \wedge b}{a}$	$\wedge$ elimination	$\dfrac{a, b}{a \wedge b}$	$\wedge$ introduction
$\dfrac{a \vdash c, b \vdash c, a \vee b}{c}$	$\vee$ elimination	$\dfrac{a}{a \vee b}, \dfrac{b}{a \vee b}$	$\vee$ introduction
$\dfrac{a, a \to b}{b}, \dfrac{a \to b, \neg b}{\neg a}$	$\to$ elimination	$\dfrac{a \vdash b}{a \to b}$	$\to$ introduction
$\dfrac{a \leftrightarrow b}{(a \to b) \wedge (b \to a)}$	$\leftrightarrow$ elimination	$\dfrac{(a \to b) \wedge (b \to a)}{a \leftrightarrow b}$	$\leftrightarrow$ elimination
$\dfrac{a, \neg a}{F}$	$\neg$ elimination	$\dfrac{a \vdash F}{\neg a}$	$\neg$ introduction
$\dfrac{F}{a}$	False elimination		
$\dfrac{\neg\neg a}{a}$	$\neg\neg$ elimination	$\dfrac{a}{\neg\neg a}$	$\neg\neg$ introduction
$\dfrac{\neg a \vdash F}{a}$	PBC	$\dfrac{}{a \vee \neg a}$	LEM

Theorems on Equivalence

$a \wedge a \equiv a$

$a \wedge \neg a \equiv F$

$a \wedge T \equiv a$

$a \wedge F \equiv F$

$\neg\neg a \equiv a$

$\neg(a \wedge b) \equiv \neg a \vee \neg b$

$a \wedge (b \vee c) \equiv (a \wedge b) \vee (a \wedge c)$

$a \rightarrow b \equiv \neg a \vee b$

$a \leftrightarrow b \equiv (a \rightarrow b) \wedge (b \rightarrow a)$

$a \vee a \equiv a$

$a \vee \neg a \equiv T$

$a \vee F \equiv a$

$a \vee T \equiv T$

$\neg(a \vee b) \equiv \neg a \wedge \neg b$

$a \vee (b \wedge c) \equiv (a \vee b) \wedge (a \vee c)$

Wumpus World Proof

1	$\neg P_{11}$	
2	$B_{11} \leftrightarrow P_{12} \vee P_{21}$	
3	$B_{21} \leftrightarrow P_{11} \vee P_{22} \vee P_{31}$	
4	$\neg B_{11}$	
5	B_{21}	
6	$(B_{11} \rightarrow P_{12} \vee P_{21}) \wedge (P_{12} \vee P_{21} \rightarrow B_{11})$	$2, \leftrightarrow E$
7	$P_{12} \vee P_{21} \rightarrow B_{11}$	$6, \wedge E$
8	$\neg(P_{12} \vee P_{21})$	$4, 7, \rightarrow E$
9	$\neg P_{12} \wedge \neg P_{21}$	8, De Morgan's
10	$\neg P_{12}$	$9, \wedge E$

Clausal Form

Conjunctive Normal Form (CNF)

- Literal: an atom or the negation of an atom
- A conjunction of disjunction of literals

$$(\neg p \vee q \vee r) \wedge (\neg q \vee r) \wedge (\neg r)$$

Conversion of a Formula to Equivalent Formula in CNF

- Every PL formula can be transformed into an equivalent PL formula in CNF.
- Steps
  1. Eliminate $\leftrightarrow$
     $P \leftrightarrow Q \equiv P \rightarrow Q \wedge Q \rightarrow P$
  2. Eliminate $\rightarrow$
     $P \rightarrow Q \equiv \neg P \vee Q$
  3. "Move $\neg$ inward" by applying De Morgan's laws
     $\neg(P \vee Q) \equiv \neg P \wedge \neg Q, \neg(P \wedge Q) \equiv \neg P \vee \neg Q$
  4. Eliminate double negation
     $\neg\neg P \equiv P$
  5. Apply distribuitive laws
     $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R), P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$

Clausal Form

- A literal is an atom (positive literal) or the negation of an atom (negative literal).
- A clause is a set of literals (implicit disjunction of its literals).
- The empty clause (an empty set of literals) denoted by $\square$, is false.
- A formula in clausal form is a set of clauses (implicit conjunction of its clauses)
- The empty set of clauses, denoted by $\emptyset$, is true.

Conversion to Clausal Form: Example

$$(p \vee r) \wedge (\neg q \vee \neg p \vee q) \wedge (p \vee \neg p \vee q \vee p \vee \neg p) \wedge (r \vee p)$$

$$\{\{p, r\}, \{\neg q, \neg p, q\}, \{p, \neg p, q\}, \{r, p\}\}$$

$$\{\{p, r\}, \{\overline{q}, \overline{p}, q\}, \{p, \overline{p}, q\}, \{r, p\}\}$$

Resolution

Resolution Rule

Resolution rule: Let C_1 , C_2 be clauses such that a literal $L \in C_1$ and its complementary literal $\overline{L} \in C_2$. The clauses C_1, C_2 are said to be clashing clauses and to clash on the complementary pair of literals $L, \overline{L}$. The resolvent of C_1 and C_2, is the clause:

$$C = Res(C_1, C_2) = (C_1 - \{L\}) \cup (C_2 - \{\overline{L}\})$$

- C_1 and C_2 are the parent clauses of C.

Resolution Rule: Example

$C_1 = \{a, b, \neg c\}, C_2 = \{b, c, \neg e\}$

Resolution Rule: Example

$$C_1 = \{a, b, \neg c\}, C_2 = \{b, c, \neg e\}$$

Resolution Rule: Example

$$C_1 = \{a, b, \neg c\}, C_2 = \{b, c, \neg e\}$$

$$C_1 - \{\neg c\} \equiv \{a, b\}, C_2 - \{c\} \equiv \{b, \neg e\}$$

Resolution Rule: Example

$$C_1 = \{a, b, \neg c\}, C_2 = \{b, c, \neg e\}$$

$$C_1 - \{\neg c\} \equiv \{a, b\}, C_2 - \{c\} \equiv \{b, \neg e\}$$

$$C \equiv C_1 - \{\neg c\} \cup C_2 - \{c\} \equiv \{a, b, \neg e\}$$

Resolution Theorem of PL

- Let K be a set of clauses.

$$Res(K) = K \cup \{R \mid R \text{ is a resolvent of two clauses in } K\}$$

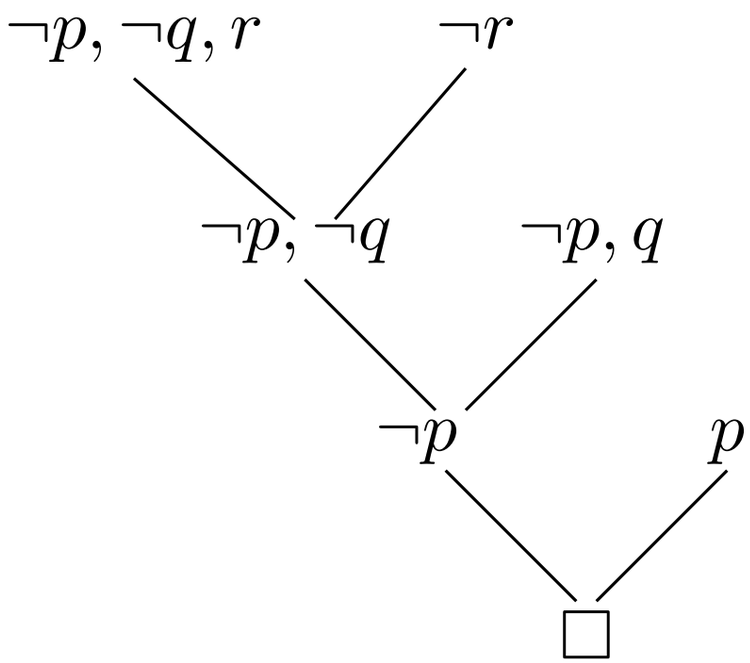
- $Res^*(K)$

$$Res^0(K) = K$$
$$Res^{n+1}(K) = Res(Res^n(K)) \text{ for } n \geq 0$$
$$Res^*(K) = Res^0(K) \cup Res^1 \cup Res^2(K) \cup \ldots = \bigcup_{n \geq 0} Res^n(K)$$

A clause set K is unsatisfiable if and only if $Res^*(K) \models \Box$.

Resolution Proof: Example

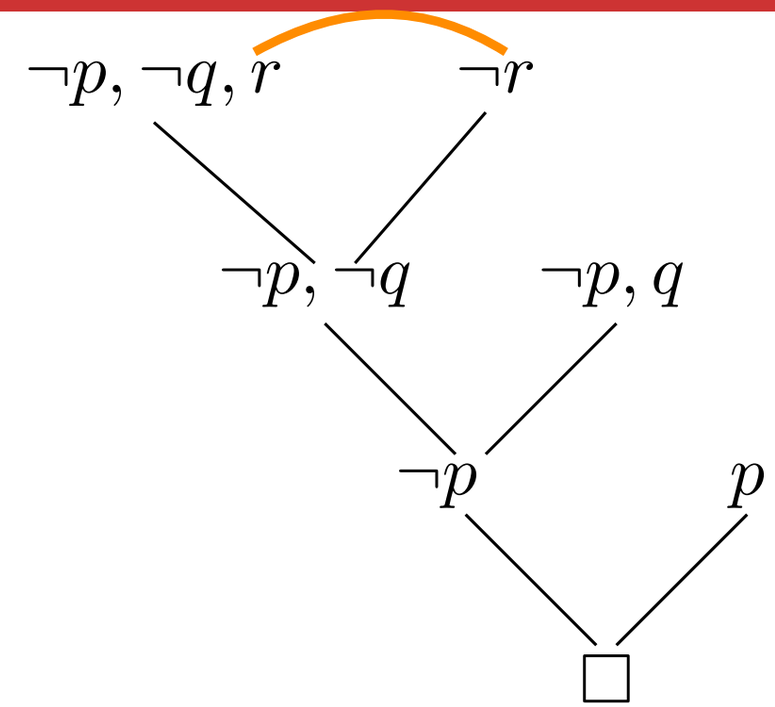
1	$\{p\}$	clause in S
2	$\{\neg p, q\}$	clause in S
3	$\{\neg r\}$	clause in S
4	$\{\neg p, \neg q, r\}$	clause in S
5	$\{\neg p, \neg q\}$	resolvent of $3, 4$
6	$\{\neg p\}$	resolvent of $2, 5$
7	$\square$	resolvent of $1, 6$



$$\dfrac{\dfrac{\dfrac{\neg p, \neg q, r \quad \neg r}{\neg p, \neg q} \quad \neg p, q}{\neg p} \quad p}{\square}$$

Resolution Proof: Example

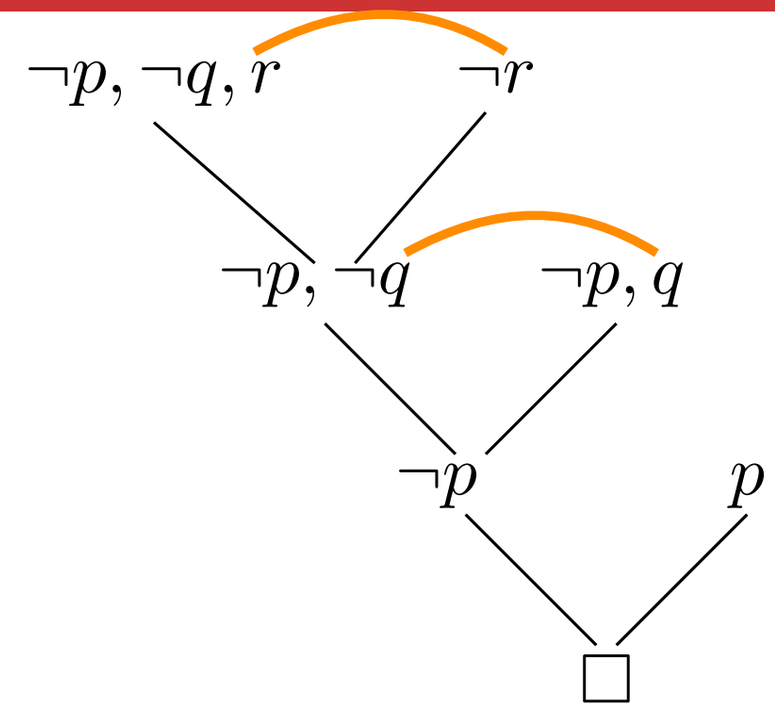
1	$\{p\}$	clause in S
2	$\{\neg p, q\}$	clause in S
3	$\{\neg r\}$	clause in S
4	$\{\neg p, \neg q, r\}$	clause in S
5	$\{\neg p, \neg q\}$	resolvent of $3, 4$
6	$\{\neg p\}$	resolvent of $2, 5$
7	$\square$	resolvent of $1, 6$



$$\dfrac{\dfrac{\dfrac{\neg p, \neg q, r \quad \neg r}{\neg p, \neg q} \quad \neg p, q}{\neg p} \quad p}{\square}$$

Resolution Proof: Example

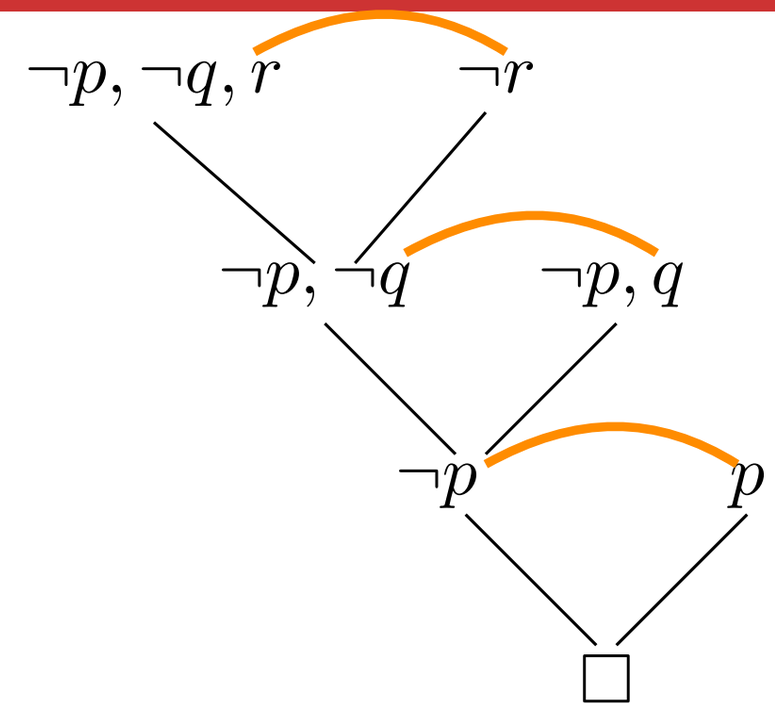
1	$\{p\}$	clause in S
2	$\{\neg p, q\}$	clause in S
3	$\{\neg r\}$	clause in S
4	$\{\neg p, \neg q, r\}$	clause in S
5	$\{\neg p, \neg q\}$	resolvent of $3, 4$
6	$\{\neg p\}$	resolvent of $2, 5$
7	$\square$	resolvent of $1, 6$



$$\dfrac{\dfrac{\dfrac{\neg p, \neg q, r \quad \neg r}{\neg p, \neg q} \quad \neg p, q}{\neg p} \quad p}{\square}$$

Resolution Proof: Example

1	$\{p\}$	clause in S
2	$\{\neg p, q\}$	clause in S
3	$\{\neg r\}$	clause in S
4	$\{\neg p, \neg q, r\}$	clause in S
5	$\{\neg p, \neg q\}$	resolvent of $3, 4$
6	$\{\neg p\}$	resolvent of $2, 5$
7	$\square$	resolvent of $1, 6$



$$\dfrac{\dfrac{\dfrac{\neg p, \neg q, r \quad \neg r}{\neg p, \neg q} \quad \neg p, q}{\neg p} \quad p}{\square}$$

Resolution Proof (or Derivation)

- A Proof (or Derivation) of the empty clause from a clause set K is a sequence $C_1, C_2, \ldots, C_m$ of clauses such that
  - for every $k = 1, \ldots, m$, C_k either is a clause in K or a resolvent of two previously derived clauses C_i, C_j with $i, j < k$.
  - the final clause C_m is the empty clause $\square$.
- To prove that a clause set K is unsatisfiable, it is enough to find a derivation of the empty clause $\square$.
- It is not necessary to find all the clauses in $Res^*(K)$.

Resolution Procedure

Algorithm: Resolution(K, G)

Input: K, set of clauses
G, query

Output: $K \cup \{\neg G\}$ is satisfiable or unsatisfiable

$C := K \cup \{\neg G\}$

loop

Choose a new pair of clashing clauses $\{C_1, C_2\} \subset C$

if no new pair exists in C **then**

return satisfiable

$R := Res(C_1, C_2)$

if $R = \square$ **then**

return unsatisfiable

$C := C \cup \{R\}$

Definite Clauses and Direct Proof

Definite Clauses

- Atoms and negated atoms are literals. An atom is a postive literal. A negated atom is a negative literal.
- A clause is a set of literals. It is a disjunction of the literals.
- A definite clause is a clause with exactly one positive literal.

$$b \rightarrow h \equiv \neg b \vee h$$

$$b_1 \wedge b_2 \wedge \ldots \wedge b_m \rightarrow h \equiv \neg b_1 \vee \neg b_2 \vee \ldots \vee \neg b_m \vee h$$

- Definite clauses cannot represent disjunction of atoms $(a \vee b)$ or disjunction of negated atoms $(\neg c \vee \neg d)$.
- A definite clause $b_1 \wedge \ldots \wedge b_m \rightarrow h$ is false in an interpretation I if $b_1, \ldots b_m$ are all true and h is false. Otherwise, it is true in I.

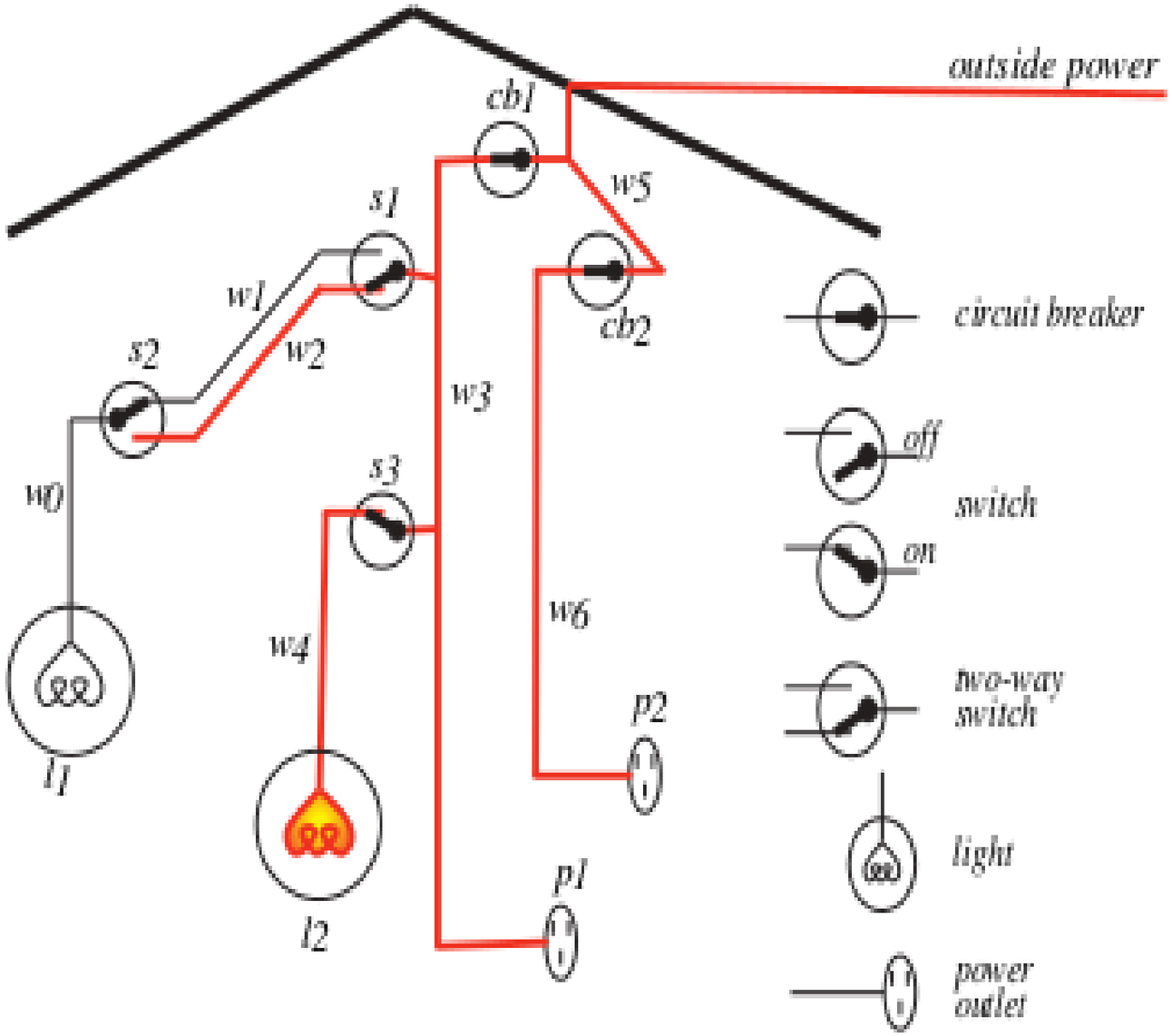
Definite Clauses

- Definite clauses are a restricted form of formulas
- A definite clause is of the form

$$b_1 \wedge \ldots \wedge b_m \rightarrow h$$

  where $h, b_1, \ldots, b_m$ are atoms.
  - h is the head of the clause
  - $b_1 \wedge \ldots \wedge b_m$ is the body of the clause (body may be empty)
- A definite clause is either a fact or a rule.
  - Rule: $b \rightarrow h$ (body and head are non-empty)
  - Fact: $\rightarrow h$, or $T \rightarrow h$, or h (body empty)
- Goal: $b \rightarrow$, or $b \rightarrow F$, or $\neg b$ (head empty) – not a definite clause.
- A knowledge base is a set of definite clauses.

Electrical Environment KB Facts



$light_l_1.$

$light_l_2.$

$down_s_1.$

$up_s_2.$

$up_s_3.$

$ok_l_1.$

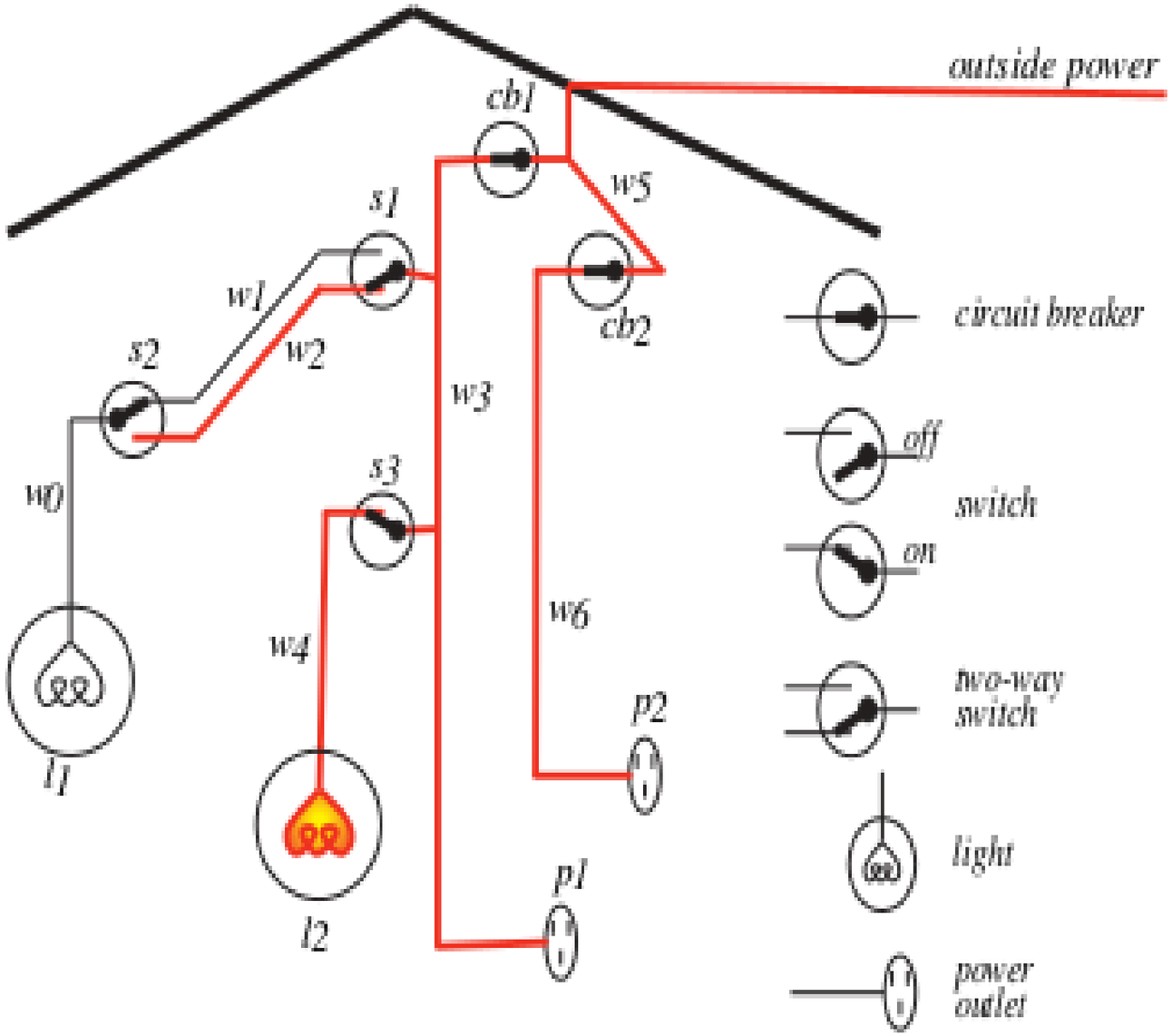
$ok_l_2.$

$ok_cb_1.$

$ok_cb_2.$

$live_outside.$

Electrical Environment KB Rules



$live_w0 \wedge ok_l_1g \rightarrow lit_l_1$

$live_w_1 \wedge up_s_2 \rightarrow live_w_0$

$live_w_2 \wedge down_s_2 \rightarrow live_w_0$

$live_w_3 \wedge up_s_1 \rightarrow live_w_1$

$live_w_3 \wedge down_s_1 \rightarrow live_w_2$

$live_w_4 \wedge ok_l_2 \rightarrow lit_l_2$

$live_w_3 \wedge up_s_3 \rightarrow live_w_4$

$live_w_3 \rightarrow live_p_1$

$live_w_5 \wedge ok_cb_1 \rightarrow live_w_3$

$live_w_6 \rightarrow live_p_2$

$live_w_5 \wedge ok_cb_2 \rightarrow live_w_6$

$live_outside \rightarrow live_w_5$

Queries and Answers

ask $light_l_1$.

answer: yes.

ask $light_l_6$.

answer: no.

ask lit_l_2

answer: yes.

Inference by Deduction

- Deduction is a form of inference.
- Deduction: determine if some proposition is a logical consequence of a knowledge base.
- A proof (or derivation) is a syntactical derivation that a proposition logically follows from a knowledge base.
- A proof procedure is a – possibly non-deterministic – algorithm for deriving a proof – logical consequences of a knowledge base.
- A proof procedure is sound if everything that can be derived from a knowledge base is a logical consequence of the knowledge base. That is, if $KB \vdash G$, then $KB \models G$.
- A proof procedure is complete if there is a proof of each logical consequence of the knowledge base. That is, if $KB \models G$, then $KB \vdash G$.

Bottom-Up Proof vs Top-Down Proof

Bottom-up	Top-down
Derive the atoms of the body first, and then select a head	Select a clause (head) first, and then derive the atoms of the body.
Data-driven	Goal-directed
Forward chaining	Backward chaining

$$\boxed{b_1 \wedge b_2 \wedge \ldots \wedge b_m} \rightarrow h$$

$$\boxed{b_1 \wedge b_2 \wedge \ldots \wedge b_m} \rightarrow h$$

Top-down Proof

Top-Down Proof

- Search top-down from a goal to determine whether it is a logical consequence of the given definite clauses.

Horn Clauses

- A definite clause is a clause with exactly one positive literal.

$$b \rightarrow h \equiv \neg b \vee h$$
$$b_1 \wedge b_2 \wedge \ldots \wedge b_m \rightarrow h \equiv \neg b_1 \vee \neg b_2 \vee \ldots \vee \neg b_m \vee h$$
$$\rightarrow h \equiv h$$

- A Horn clause is a clause with at most one positive literal.

$$b \rightarrow h \equiv \neg b \vee h$$
$$b_1 \wedge b_2 \wedge \ldots \wedge b_m \rightarrow h \equiv \neg b_1 \vee \neg b_2 \vee \ldots \vee \neg b_m \vee h$$
$$\rightarrow h \equiv h$$
$$b \rightarrow \equiv \neg b$$
$$b_1 \wedge b_2 \wedge \ldots \wedge b_m \rightarrow \equiv \neg b_1 \vee \neg b_2 \vee \ldots \vee \neg b_m$$

Horn Clauses

- Horn clauses are definite clauses which allow empty heads in the clauses.
- A Horn clause is of the form

$$h \leftarrow b_1 \wedge \ldots \wedge b_m$$

where $h, b_1, \ldots, b_m$ are atoms.

$b_1 \wedge \ldots \wedge b_m$	$\rightarrow h$			Rule (definite clause
	$\rightarrow h$	T	$\rightarrow h$	Fact
$b_1 \wedge \ldots \wedge b_m$	$\rightarrow$	$b_1 \wedge \ldots \wedge b_m$	$\rightarrow F$	Goal clause, negative clause

$b_1 \wedge \ldots \wedge b_m$	$\rightarrow h$			$\{h, \neg b_1, \ldots, \neg b_m\}$
	$\rightarrow h$	T	$\rightarrow h$	$\{h\}$
$b_1 \wedge \ldots \wedge b_m$	$\rightarrow$	$b_1 \wedge \ldots \wedge b_m$	$\rightarrow F$	$\{\neg b_1, \ldots, \neg b_m\}$

Resolution Rule

- Given the goal $b_1 \wedge b_2 \wedge \ldots \wedge b_m \rightarrow$, select an atom, say b_1, in its body — subgoal.
- Resolution steps
  - Choose a definite clause in KB with b_1 as the head (b_1-clause).

    $$a_1 \wedge a_2 \wedge \ldots \wedge a_p \rightarrow b_1$$

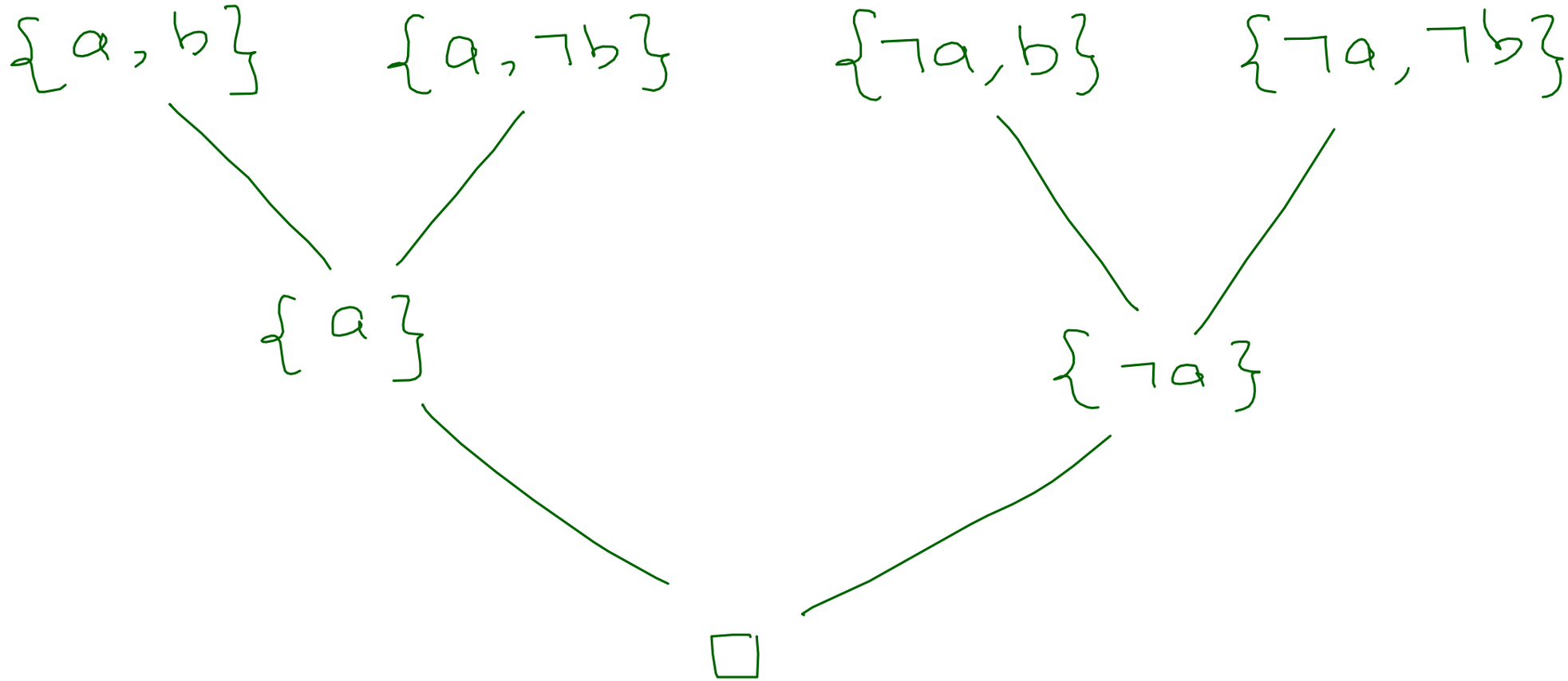
    If there is no such clause, fail.
  - Resolve the two clauses into the resolvent

    $$\begin{array}{lll} b_1 \wedge b_2 \wedge \ldots \wedge b_m & \rightarrow & \{\neg b_1, \neg b_2, \ldots, \neg b_m\} \\ a_1 \wedge \ldots \wedge a_p & \rightarrow b_1 & \{\neg a_1, \ldots, \neg a_p, b_1\} \\ a_1 \wedge \ldots \wedge a_p \wedge b_2 \ldots \wedge b_m & \rightarrow & \{\neg a_1, \ldots, \neg a_p, \neg b_2, \ldots, \neg b_m\} \end{array}$$

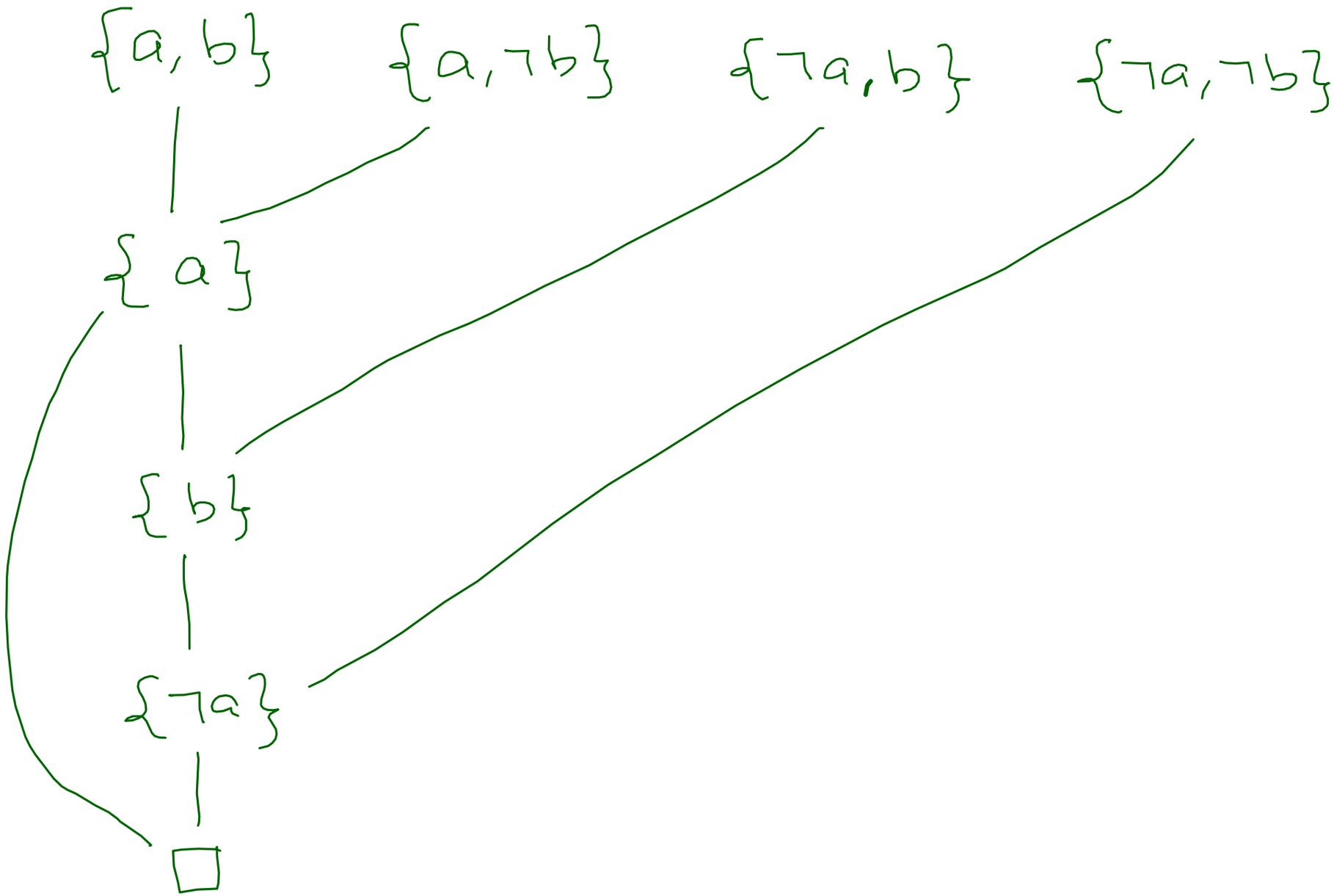
    An atom is replaced by the body of a definite clause which has that atom in the head.
- Derive: $\rightarrow$

Non-linear Resolution: Example

$K = \{ \{a, b\}, \{a, \neg b\}, \{\neg a, b\}, \{\neg a, \neg b\} \}$

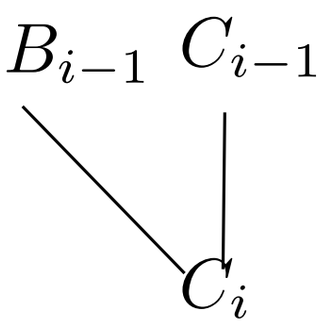


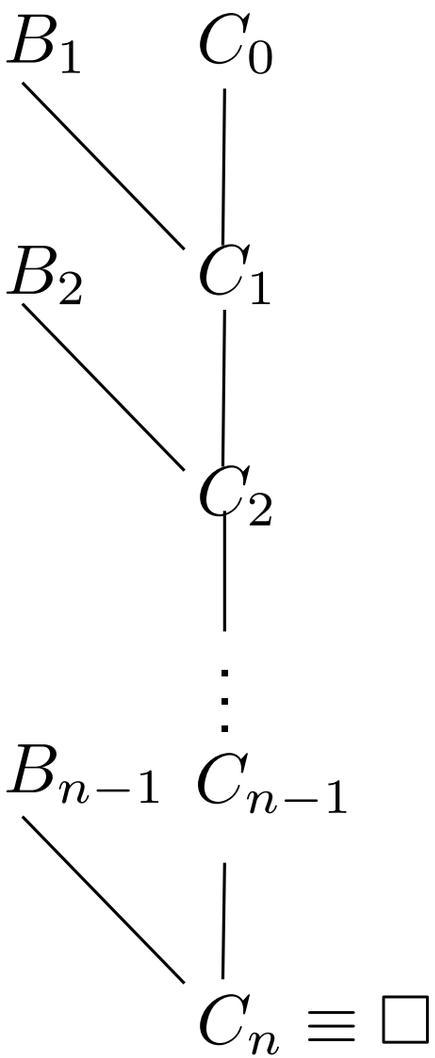
Linear Resolution: Example



Linear Resolution

- Start with a base clause C in K
- Derive a sequence of clauses $C_0, C_1, \ldots, C_n$ such that $C_0 = C, C_n = \square$
- Each clause C_i is derived by resolving the preceding clause C_{i-1} (main clause) with another clause B_{i-1} (side-caluse). B_{i-1} is is either an input clause in K or an earlier clause C_j for some $j < i$.
- Linear resoution is complete: For every unsatisfiable clause set K, there is a base clause $C \in K$ from which the empty clause can be derived by linear resolution.

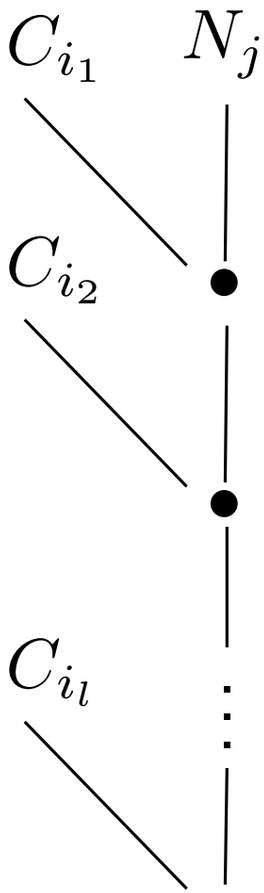




SLD Resolution

- K be a set of Horn clauses $\{C_1, C_2, \ldots, C_n, N_1, N_2, \ldots, N_m\}$
- $C_1, C_2, \ldots, C_n$ are definite clauses and $N_1, \ldots, N_m$ are goal clauses (negative clauses).
- Main clause is a goal clause (negative clause)
- Side-clause is a definite clause from the KB.
- A goal clause and a definite clause resolve into a goal clause.

$$
\begin{array}{ll}
b_1 \wedge b_2 \wedge \ldots \wedge b_m & \rightarrow \\
a_1 \wedge a_2 \wedge \ldots \wedge a_p & \rightarrow b_1 \\
a_1 \wedge a_2 \wedge \ldots \wedge a_p \wedge b_2 \ldots \wedge b_m & \rightarrow
\end{array}
$$



- SLD resolution: Linear resolution with Selection function for Definite clauses
  - S: Selection function (to select the next definite clause)
  - L: Linear resolution
  - D: Definite clauses.

LUSH Resolution

- LUSH = Linear resolution with Unrestricted Selection for Horn clauses
- Add the goal clause $g_1 \wedge \ldots \wedge g_k \rightarrow$ to KB.
- A proof of the goal clause is a sequence of goal clauses $D_0, D_1, \ldots, D_n$:
  - Select a goal clause $N_j, j \in \{1, \ldots, m\}$
  - Choose a sequence of clauses $C_{i_1}, C_{i_2}, \ldots$ such that $i_1, i_2, \ldots, i_l \in \{1, \ldots, n\}$ such that D_{j+1} is the resolvent of D_j with a clause C_{i_j} in KB.
  - D_n is an empty clause.

- Maintains a set G of atoms to prove (subgoals).
- Initially, G contains the atoms in the goal.
- A clause $b_1 \wedge \ldots \wedge b_p \rightarrow a$ means a can be replaced by $b_1, \ldots, b_p$.
- The goal is proved when G becomes empty.

Top-Down Proof: Example

1	$b \wedge c \rightarrow a$
2	$d \wedge e \rightarrow b$
3	$g \wedge e \rightarrow b$
4	$e \rightarrow c$
5	$\rightarrow d$
6	$\rightarrow e$
7	$a \wedge g \rightarrow f$

8	$a \rightarrow$	
9	$b \wedge c \rightarrow$	8, 1
10	$d \wedge e \wedge c \rightarrow$	9, 2
11	$e \wedge c \rightarrow$	10, 5
12	$c \rightarrow$	11, 6
13	$e \rightarrow$	12, 4
14	$\rightarrow$	13, 6

Top-Down Proof Procedure

Algorithm: Prove_DC_TD(KB, Goal):non-deterministic

Input: KB: set of definite clauses
Goal: set of atoms to prove

Output: yes if $KB \vdash$ Goal; fails otherwise

Local: G, goal set, set of sub-goal atoms

$G :=$ Goal
repeat
  select an atom a in G
  choose a definite clause $B \rightarrow a$ in KB with a as head `// non-deterministic`
  $G := (G \setminus \{a\}) \cup B$
until $G = \{\}$
return yes

Selection of an Atom and Choice of a Clause

- "Don't care" non-determinism
  - Select any atom in the body.
  - If one selection does not lead to proof, no other selection will lead to proof
  - Atoms can be evaluated in any order.
  - Selection function example: select the leftmost atom – canonical evaluation.
- "Don't know" non-determinism
  - Choose: Search for a clause until the proof succeeds.
  - If one choice does not lead to proof, other choices may lead to proof
  - Search the whole computation tree until a first solution is found.

Search Graph for SLD Resolution

$a \rightarrow b \wedge c.$ $\quad a \rightarrow g.$ $\quad a \rightarrow h.$

$b \rightarrow j.$ $\quad b \rightarrow k.$

$d \rightarrow p.$ $\quad d \rightarrow m.$

$f \rightarrow m.$ $\quad f \rightarrow p.$

$g \rightarrow m.$ $\quad g \rightarrow f.$

$h \rightarrow m.$ $\quad k \rightarrow m.$

$p.$

Search Graph for SLD Resolution

$a \rightarrow b \wedge c.$ $\quad a \rightarrow g.$ $\quad a \rightarrow h.$

$b \rightarrow j.$ $\quad b \rightarrow k.$

$d \rightarrow p.$ $\quad d \rightarrow m.$

$f \rightarrow m.$ $\quad f \rightarrow p.$

$g \rightarrow m.$ $\quad g \rightarrow f.$

$h \rightarrow m.$ $\quad k \rightarrow m.$

$p.$

$a \wedge d \rightarrow$

Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$ $\quad a \to g. \quad a \to h.$

$b \to j. \quad b \to k.$

$d \to p. \quad d \to m.$

$f \to m. \quad f \to p.$

$g \to m. \quad g \to f.$

$h \to m. \quad k \to m.$

$p.$

$a \wedge d \to$

$b \wedge c \wedge d \to$

Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$	$a \to g.$	$a \to h.$
$\boxed{b \to j.}$	$b \to k.$	
$d \to p.$	$d \to m.$	
$f \to m.$	$f \to p.$	
$g \to m.$	$g \to f.$	
$h \to m.$	$k \to m.$	
$p.$		

$a \wedge d \to$

$b \wedge c \wedge d \to$

$j \wedge c \wedge d \to$

Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$	$a \to g.$	$a \to h.$
$\boxed{b \to j.}$	$\boxed{b \to k.}$	
$d \to p.$	$d \to m.$	
$f \to m.$	$f \to p.$	
$g \to m.$	$g \to f.$	
$h \to m.$	$k \to m.$	
$p.$		

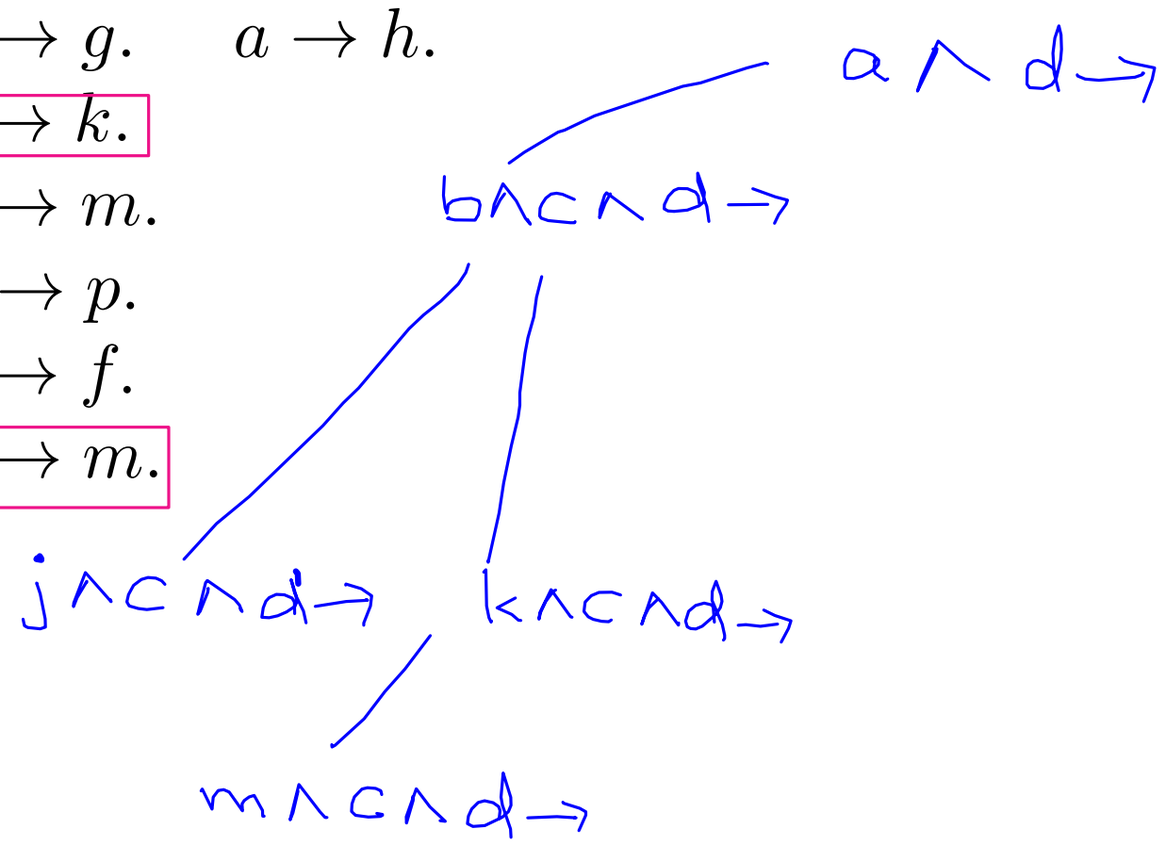
$a \wedge d \to$

$b \wedge c \wedge d \to$

$j \wedge c \wedge d \to$ $\quad$ $k \wedge c \wedge d \to$

Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$	$a \to g.$	$a \to h.$
$\boxed{b \to j.}$	$\boxed{b \to k.}$	
$d \to p.$	$d \to m.$	
$f \to m.$	$f \to p.$	
$g \to m.$	$g \to f.$	
$h \to m.$	$\boxed{k \to m.}$	
$p.$		



Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$ $\boxed{a \to g.}$ $a \to h.$

$\boxed{b \to j.}$ $\boxed{b \to k.}$

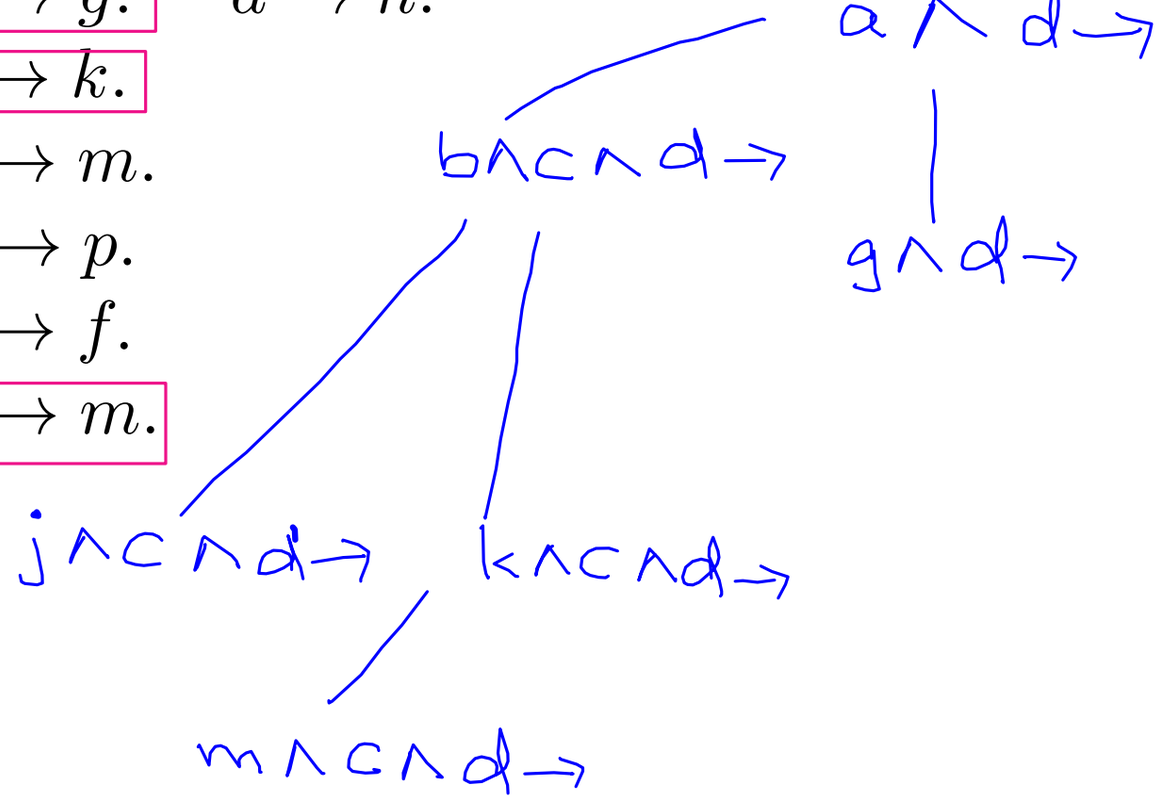
$d \to p.$ $d \to m.$

$f \to m.$ $f \to p.$

$g \to m.$ $g \to f.$

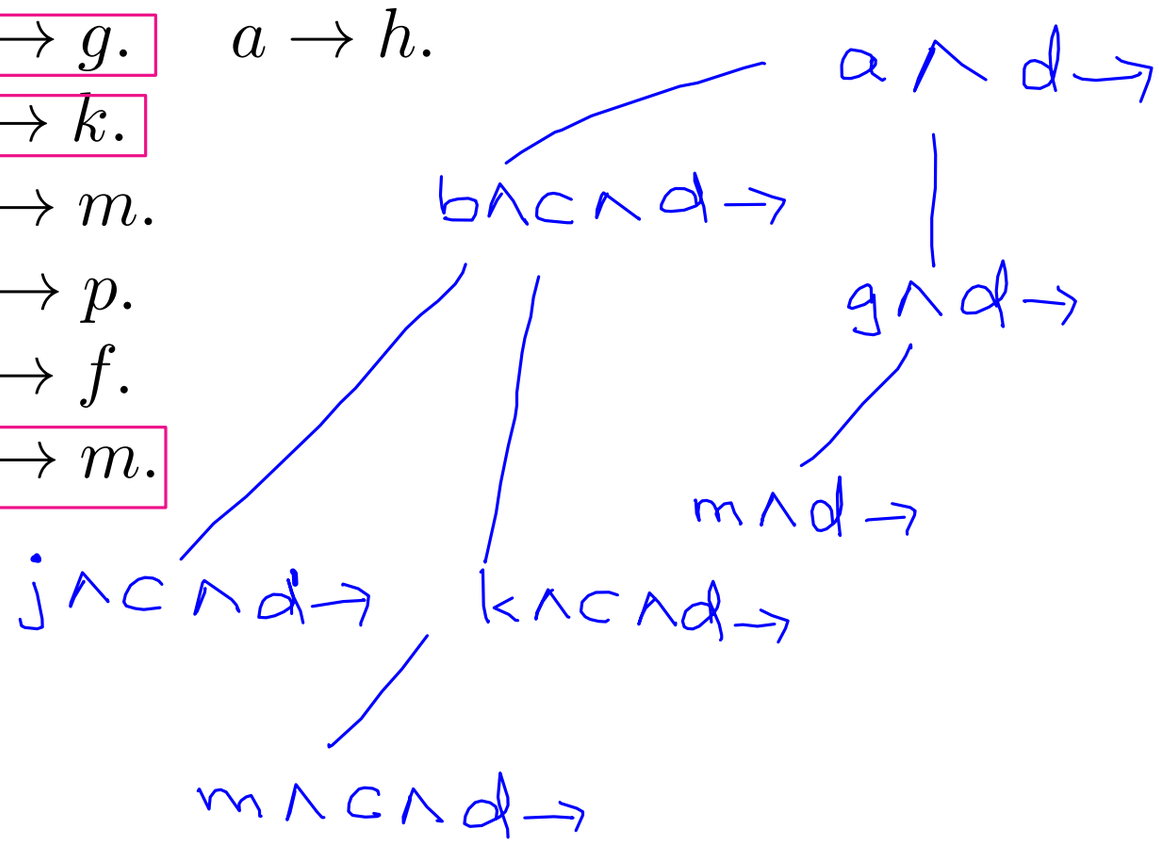
$h \to m.$ $\boxed{k \to m.}$

$p.$



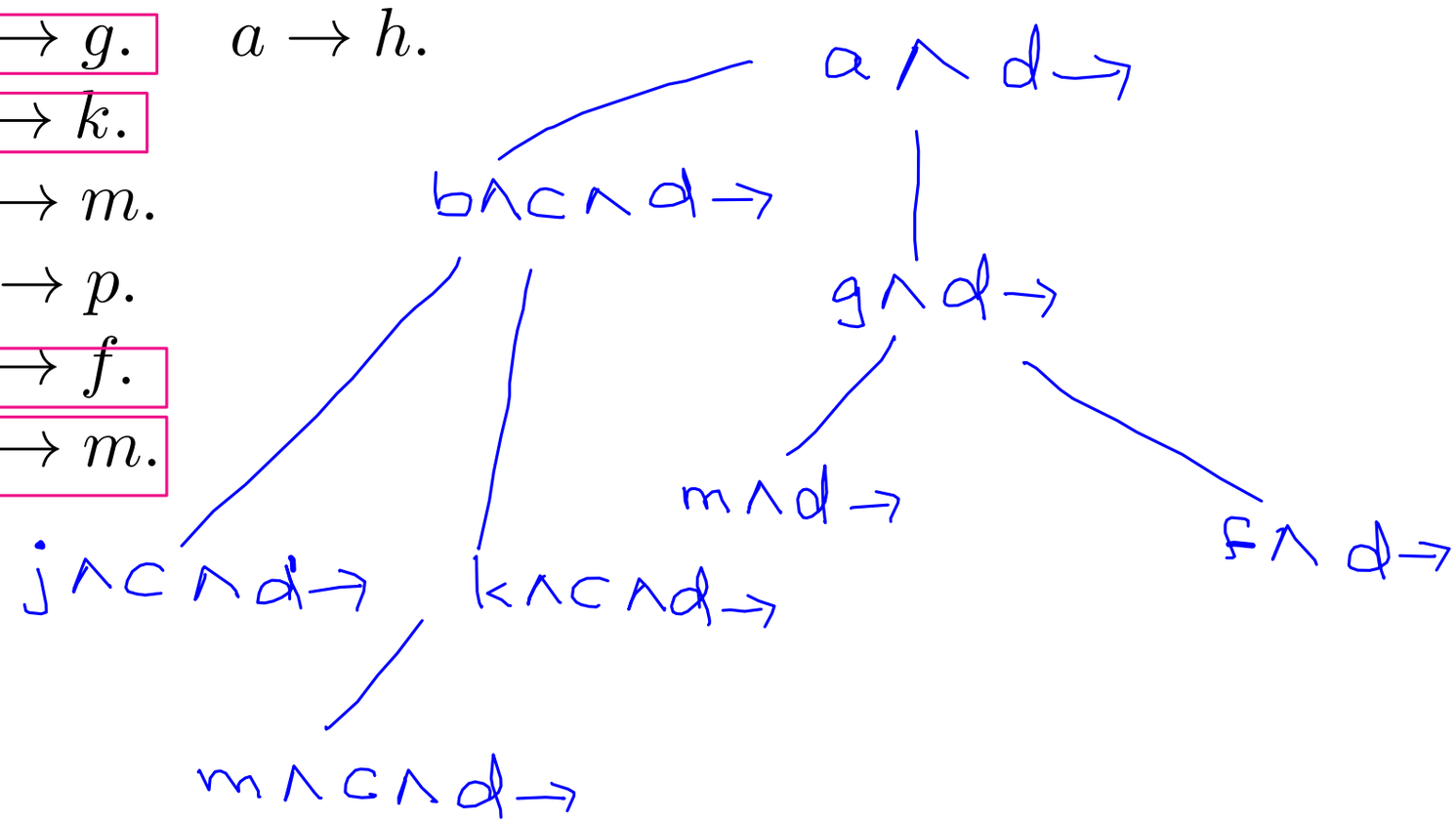
Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$	$\boxed{a \to g.}$	$a \to h.$
$\boxed{b \to j.}$	$\boxed{b \to k.}$	
$d \to p.$	$d \to m.$	
$f \to m.$	$f \to p.$	
$\boxed{g \to m.}$	$g \to f.$	
$h \to m.$	$\boxed{k \to m.}$	
$p.$		



Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$	$\boxed{a \to g.}$	$a \to h.$
$\boxed{b \to j.}$	$\boxed{b \to k.}$	
$d \to p.$	$d \to m.$	
$f \to m.$	$f \to p.$	
$\boxed{g \to m.}$	$\boxed{g \to f.}$	
$h \to m.$	$\boxed{k \to m.}$	
$p.$		



Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$ $\boxed{a \to g.}$ $a \to h.$

$\boxed{b \to j.}$ $\boxed{b \to k.}$

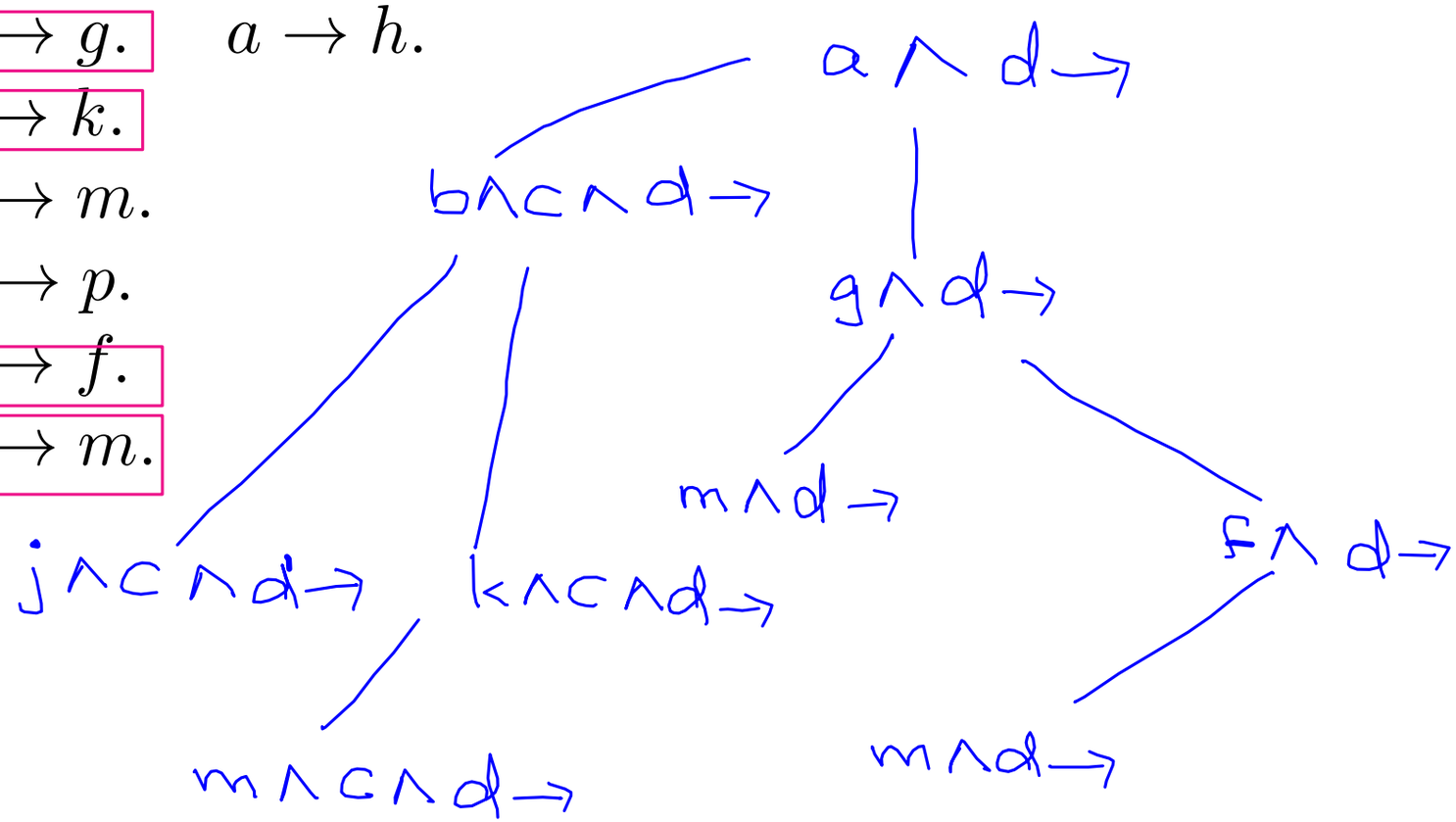
$d \to p.$ $d \to m.$

$\boxed{f \to m.}$ $f \to p.$

$\boxed{g \to m.}$ $\boxed{g \to f.}$

$h \to m.$ $\boxed{k \to m.}$

$p.$



Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$ $\boxed{a \to g.}$ $a \to h.$

$\boxed{b \to j.}$ $\boxed{b \to k.}$

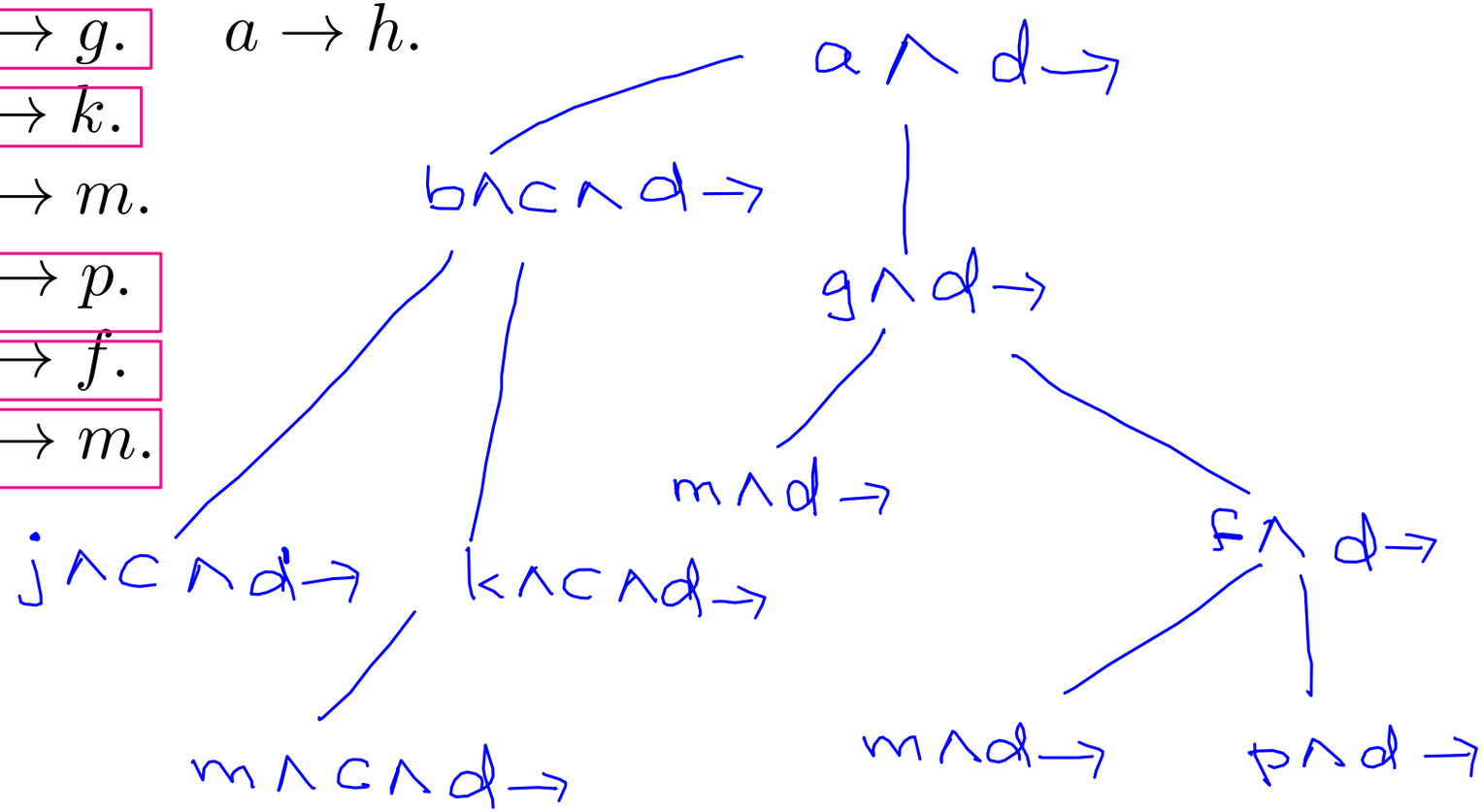
$d \to p.$ $d \to m.$

$\boxed{f \to m.}$ $\boxed{f \to p.}$

$\boxed{g \to m.}$ $\boxed{g \to f.}$

$h \to m.$ $\boxed{k \to m.}$

$p.$



Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$ $\boxed{a \to g.}$ $a \to h.$

$\boxed{b \to j.}$ $\boxed{b \to k.}$

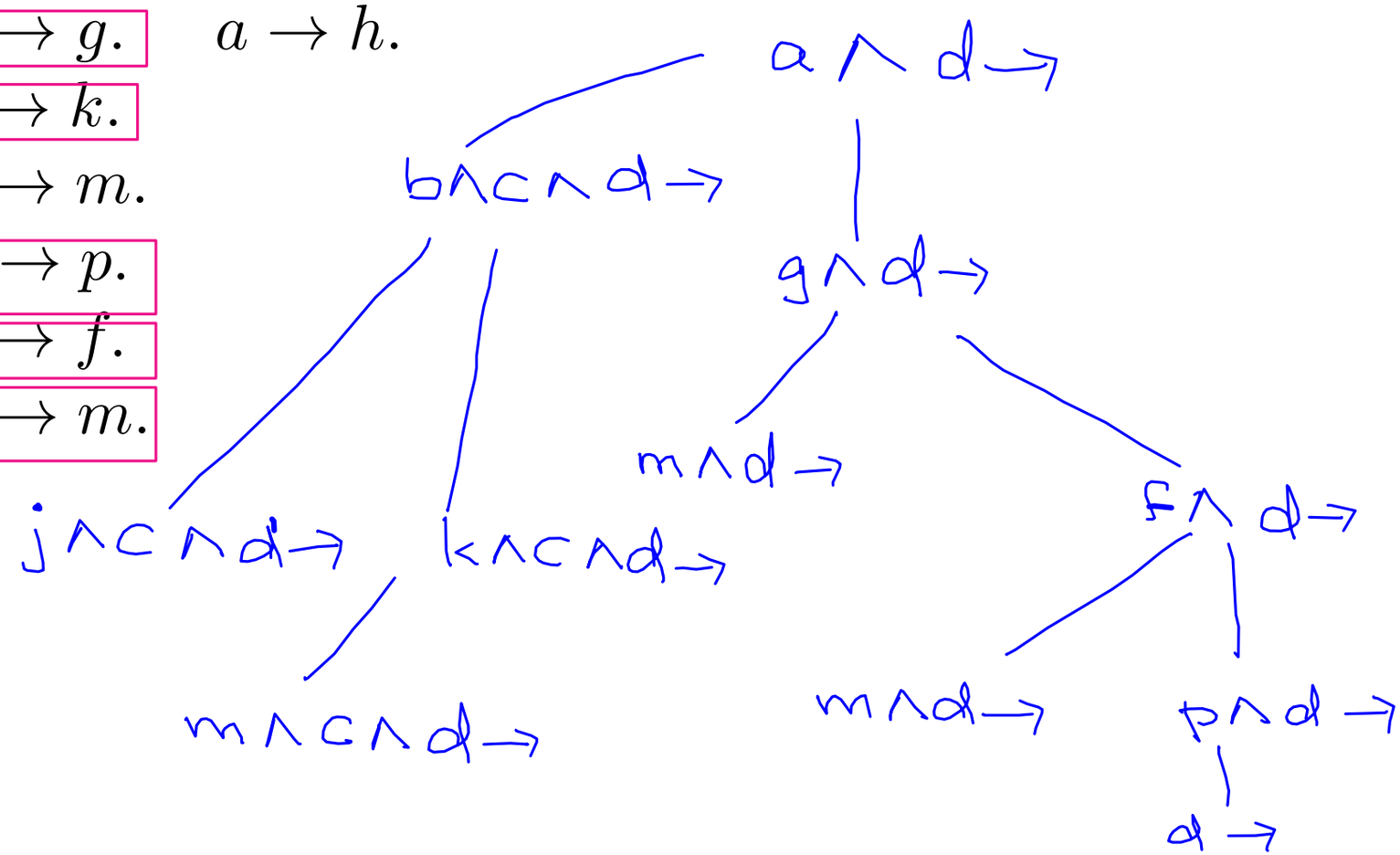
$d \to p.$ $d \to m.$

$\boxed{f \to m.}$ $\boxed{f \to p.}$

$\boxed{g \to m.}$ $\boxed{g \to f.}$

$h \to m.$ $\boxed{k \to m.}$

$\boxed{p.}$



Search Graph for SLD Resolution

$\boxed{a \to b \wedge c.}$ $\boxed{a \to g.}$ $a \to h.$

$\boxed{b \to j.}$ $\boxed{b \to k.}$

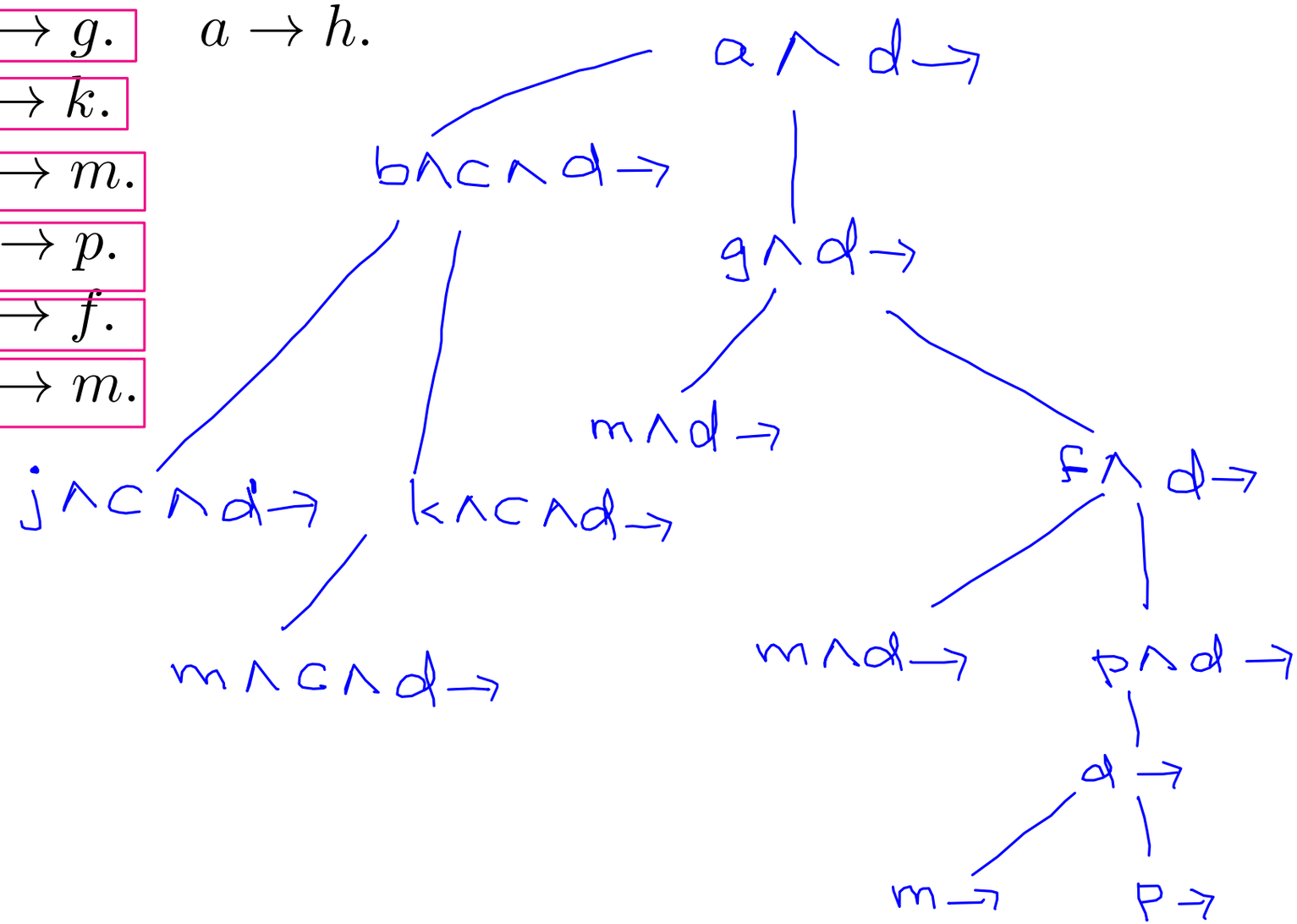
$\boxed{d \to p.}$ $\boxed{d \to m.}$

$\boxed{f \to m.}$ $\boxed{f \to p.}$

$\boxed{g \to m.}$ $\boxed{g \to f.}$

$h \to m.$ $\boxed{k \to m.}$

$\boxed{p.}$



Search Graph for SLD Resolution

$a \to b \wedge c.$ $\quad a \to g.$ $\quad a \to h.$

$b \to j.$ $\quad b \to k.$

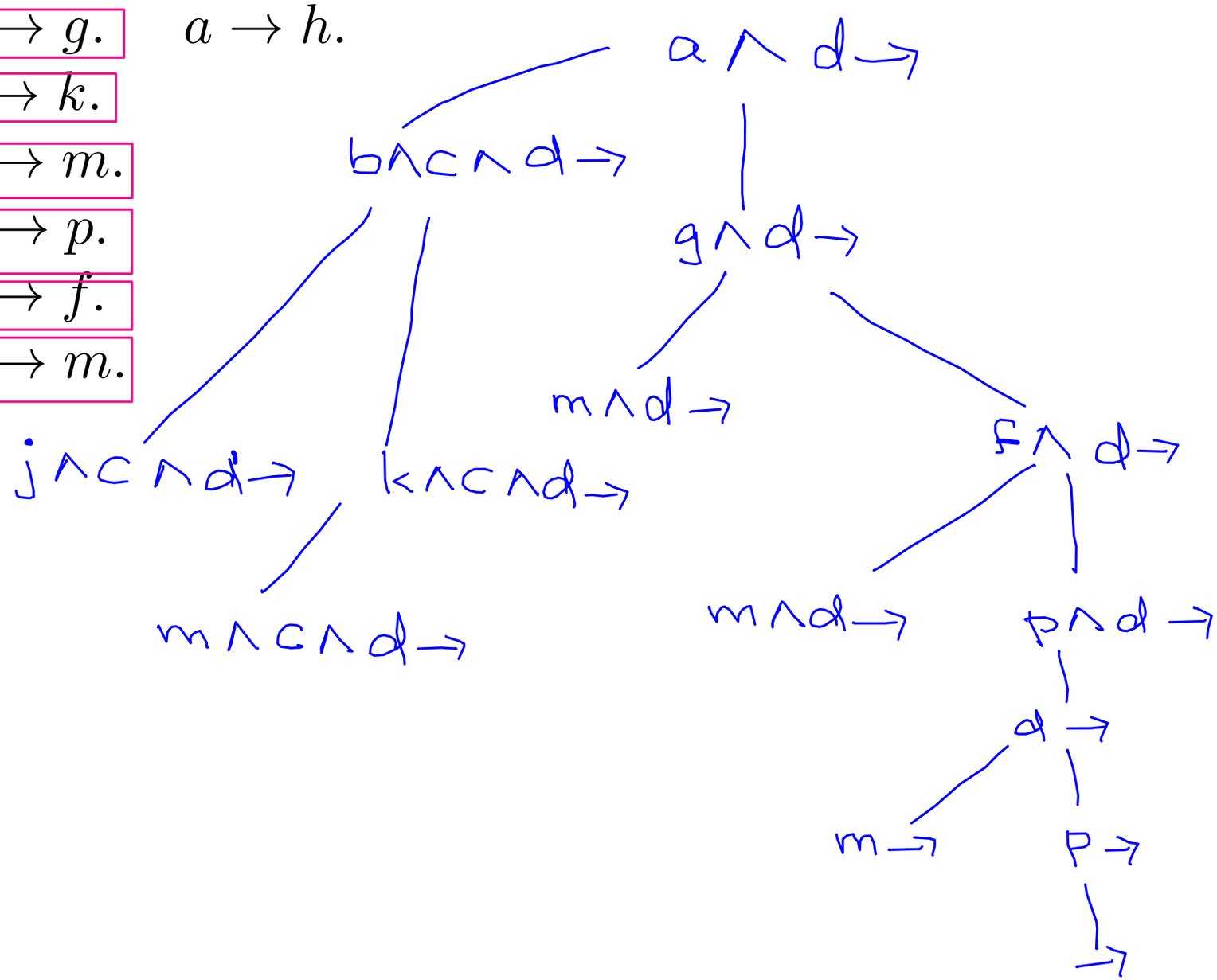
$d \to p.$ $\quad d \to m.$

$f \to m.$ $\quad f \to p.$

$g \to m.$ $\quad g \to f.$

$h \to m.$ $\quad k \to m.$

$p.$



Search Graph for SLD Resolution

$a \to b \wedge c.$ $a \to g.$ $a \to h.$

$b \to j.$ $b \to k.$

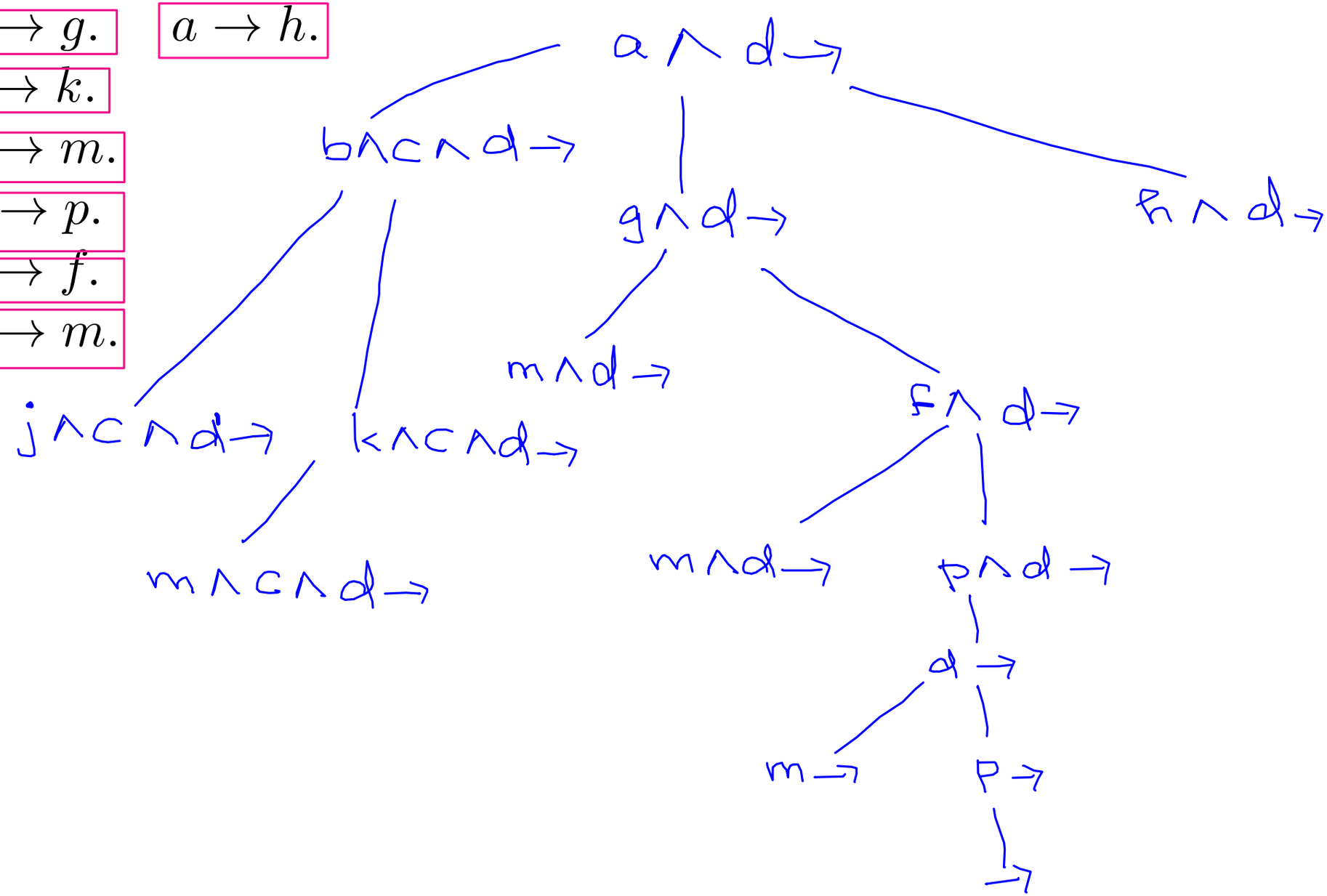
$d \to p.$ $d \to m.$

$f \to m.$ $f \to p.$

$g \to m.$ $g \to f.$

$h \to m.$ $k \to m.$

$p.$



Search Graph for SLD Resolution

$a \rightarrow b \wedge c.$ $a \rightarrow g.$ $a \rightarrow h.$

$b \rightarrow j.$ $b \rightarrow k.$

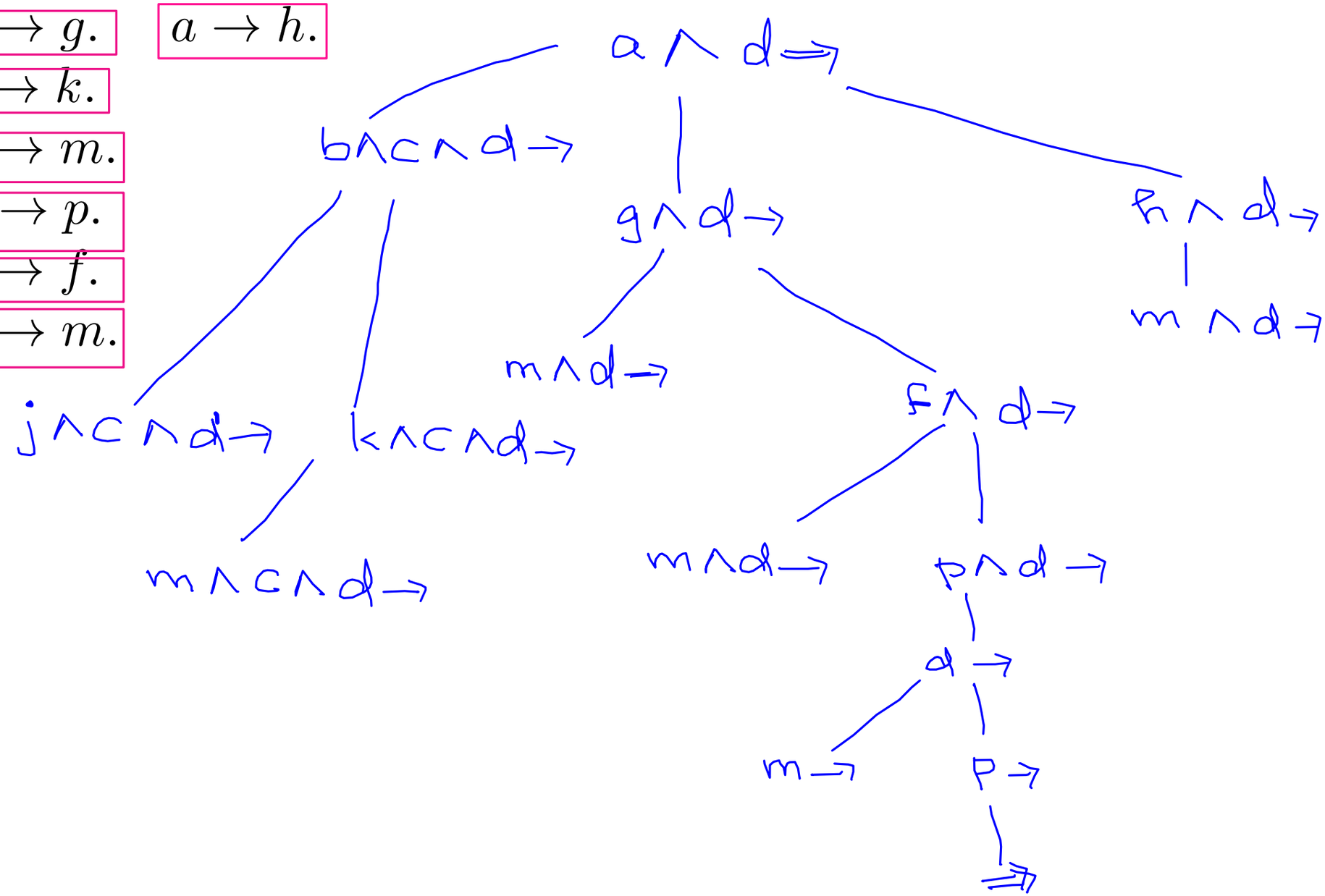
$d \rightarrow p.$ $d \rightarrow m.$

$f \rightarrow m.$ $f \rightarrow p.$

$g \rightarrow m.$ $g \rightarrow f.$

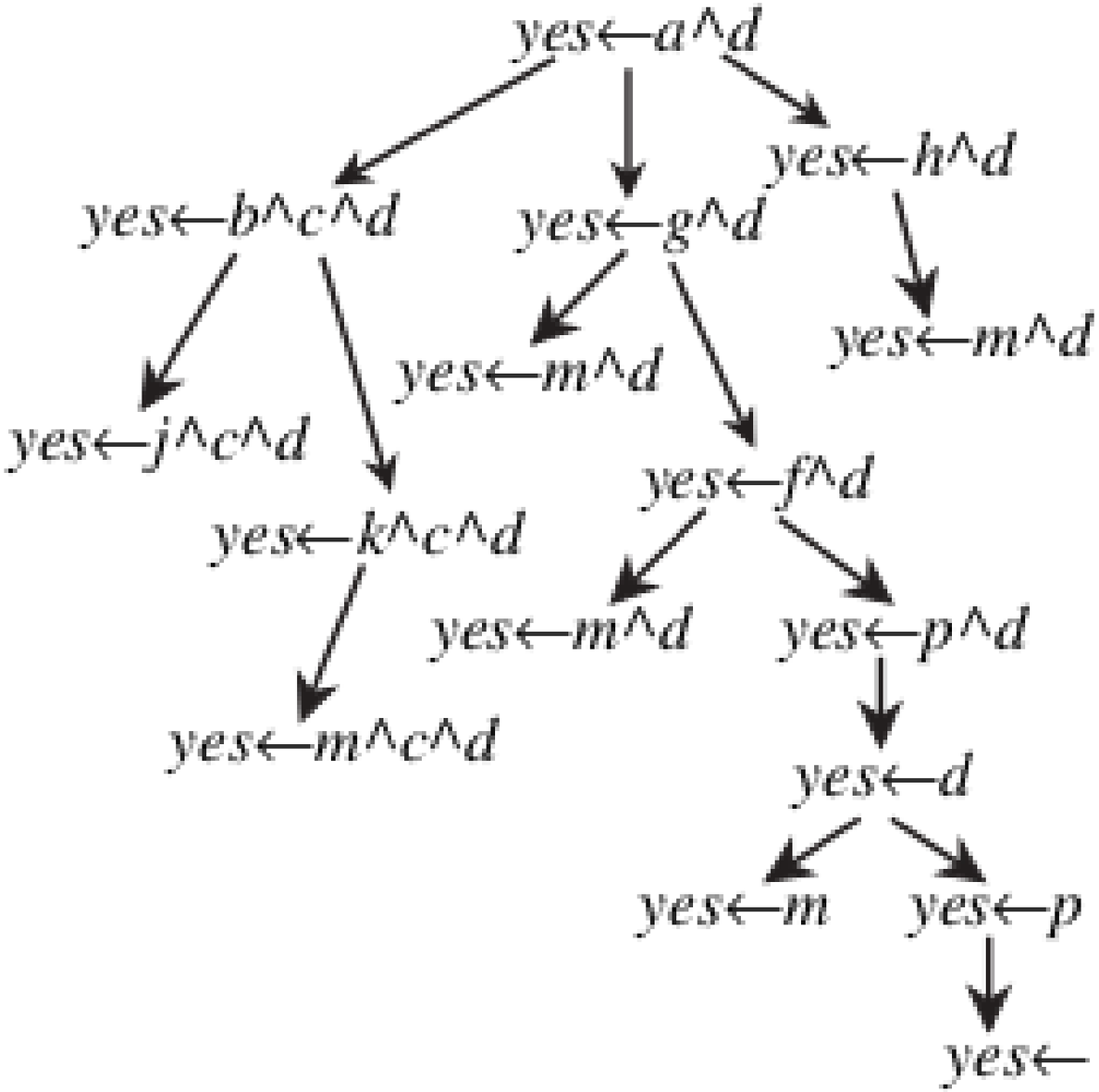
$h \rightarrow m.$ $k \rightarrow m.$

$p.$



Search Graph for SLD Resolution

$a \rightarrow b \wedge c.$ $\quad a \rightarrow g.$ $\quad a \rightarrow h.$
$b \rightarrow j.$ $\quad b \rightarrow k.$
$d \rightarrow p.$ $\quad d \rightarrow m.$
$f \rightarrow m.$ $\quad f \rightarrow p.$
$g \rightarrow m.$ $\quad g \rightarrow f.$
$h \rightarrow m.$ $\quad k \rightarrow m.$
$p.$



SLD Resolution: Example

Cosntruct a SLD resolution proof to deduce the goal "Insurance rates will decrease." from the following Knowledge Base:

- If it is raining, then the ground is wet.
- If the ground is wet, then the grass is slippery.
- If the grass is slippery, then people will walk carefully.
- It is raining.
- If people walk carefully, then accidents will be fewer.
- If accidents are fewer, then insurance rates will decrease.

SLD Resolution: Example

Suppose we have the apparatuses available to perform the following chemical reactions.

$$MgO + H_2 \rightarrow Mg + H_2O$$
$$C + O2 \rightarrow CO_2$$
$$H_2O + CO_2 \rightarrow H_2CO_3$$

Further, our lab has the following basic materials available: MgO, H_2, O_2 and C. Prove that under these circumstances it is possible to produce H_2CO_3.

Bottom-up Proof

Bottom-Up Inference Rule

- Derive all logical consequences from the KB.
- Inference rule: $\rightarrow$ elimination (modus ponens)

$$\frac{b, \quad b \rightarrow h}{h}$$

- If $b_1 \wedge \ldots \wedge b_m \rightarrow g$ is a definite clause in the knowledge base, and each b_i has been derived, then h can be derived.

$$\frac{b_1 \wedge \ldots \wedge b_m, \quad b_1, \ldots, b_m \rightarrow h}{h}$$

Bottom-Up Proof: Example

1	$b \wedge c$	$\rightarrow a$		C, consequence set	
2	$d \wedge e$	$\rightarrow b$	8	$\{\}$	
3	$g \wedge e$	$\rightarrow b$	9	$\{d\}$	5
4	e	$\rightarrow c$	10	$\{e, d\}$	$6, 9$
5		$\rightarrow d$	11	$\{c, e, d\}$	$4, 10$
6		$\rightarrow e$	12	$\{b, c, e, d\}$	$2, 11$
7	$a \wedge g$	$\rightarrow f$	13	$\{a, b, c, e, d\}$	$1, 12$

$$KB \vdash a \quad KB \vdash b$$

$$KB \nvdash f \quad KB \nvdash g$$

Bottom-Up Proof Procedure

Algorithm: Prove_DC_BU(KB)

Input: KB, set of definite clauses
Output: Set of all atoms that are logical consequences of KB
Local: C, consequence set, the set of consequence atoms

$C := \{\}$
repeat
    select $b_1 \wedge \ldots \wedge b_m \rightarrow h$ in KB where $b_i \in C$ for all i, and $h \notin C$
    $C := C \cup \{h\}$
until no more clauses can be selected
return C

- When the proof procedure terminates
  - Case goal g is an atom: if $g \in C$, then $KB \vdash g$, or
  - Case goal is a conjunction $g_1 \wedge \ldots \wedge g_k$: if $\{g_1, \ldots, g_k\} \subseteq C$), then $KB \vdash g_1 \wedge \ldots \wedge g_k$.

AND-OR Graph

1 $q \leftarrow p$

2 $p \leftarrow l \wedge m$

3 $m \leftarrow b \wedge l$

4 $l \leftarrow a \wedge p$

5 $l \leftarrow a \wedge b$

6 a

7 b

AND-OR Graph

1 $q \leftarrow p$

2 $p \leftarrow l \wedge m$

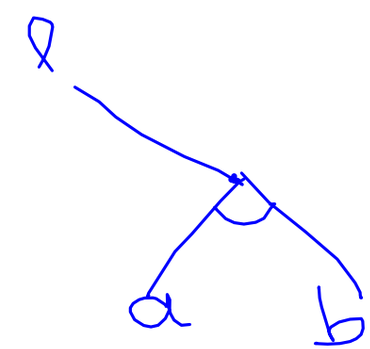
3 $m \leftarrow b \wedge l$

4 $l \leftarrow a \wedge p$

5 $l \leftarrow a \wedge b$

6 a

7 b



AND-OR Graph

1 $q \leftarrow p$

2 $p \leftarrow l \wedge m$

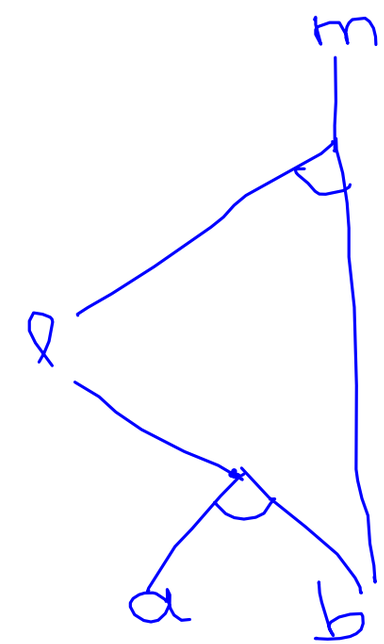
3 $m \leftarrow b \wedge l$

4 $l \leftarrow a \wedge p$

5 $l \leftarrow a \wedge b$

6 a

7 b



AND-OR Graph

1 $q \leftarrow p$

2 $p \leftarrow l \wedge m$

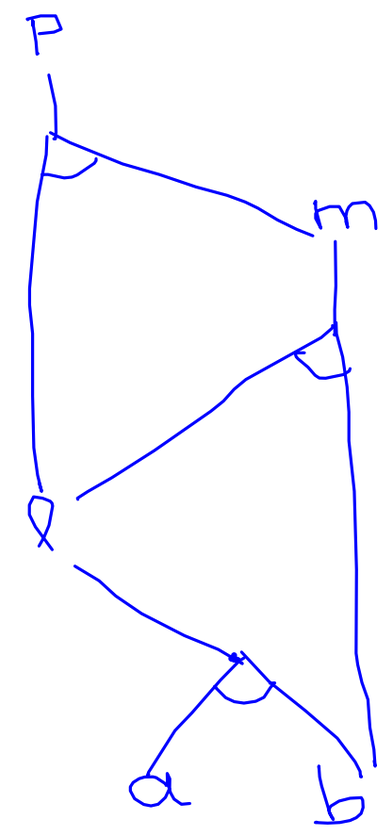
3 $m \leftarrow b \wedge l$

4 $l \leftarrow a \wedge p$

5 $l \leftarrow a \wedge b$

6 a

7 b



AND-OR Graph

1 $q \leftarrow p$

2 $p \leftarrow l \wedge m$

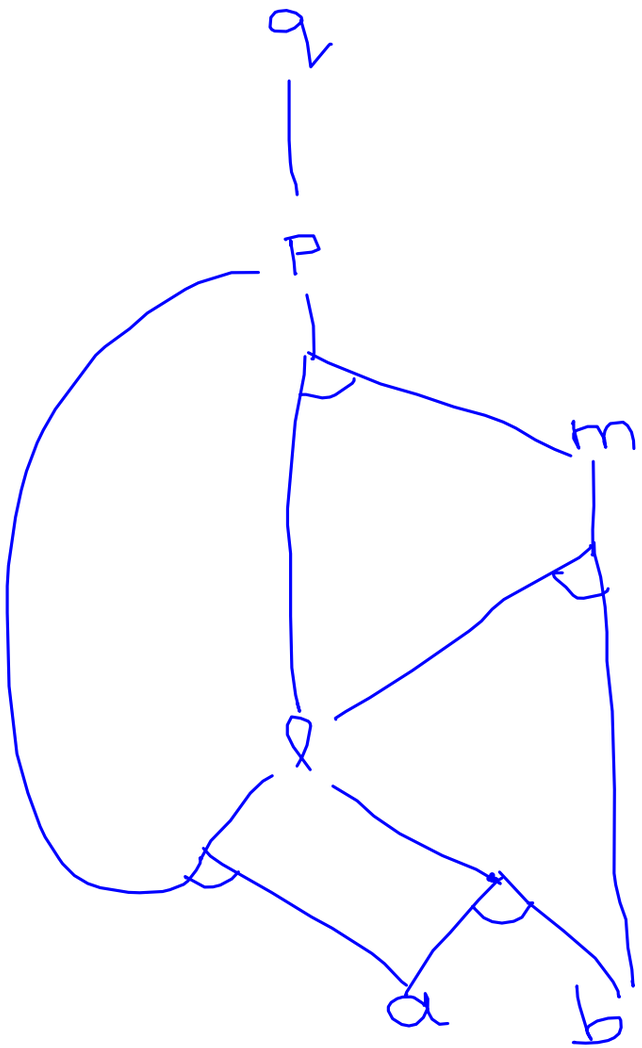
3 $m \leftarrow b \wedge l$

4 $l \leftarrow a \wedge p$

5 $l \leftarrow a \wedge b$

6 a

7 b



Bottom-Up Proof is Sound

Every atom in C is a logical consequence of KB (If $KB \vdash g$ then $KB \models g$).

1	Assume there is an atom in C that is not a logical consequence of KB. (PBC)	
2	Let h be the first atom added to C, which is not a logical consequence of KB.	
3	There is a model I of KB in which h is false.	1
4	h is in C, so there is a clause in KB of the form $h \leftarrow a_1 \wedge \ldots \wedge a_m$ where $a_1, a_2, \ldots, a_m$ are all in C.	
5	$a_1, a_2, \ldots, a_m$ are true in I.	2
6	For $h \leftarrow a_1 \wedge a_2 \wedge \ldots \wedge a_m$, the body is true in I, but the head is false in I.	3, 5
7	Therefore, this clause is false in I.	
8	This is a contradiction to "I is a model of KB".	3
9	Thus, every atom of C is a logical consequence of KB.	

Bottom-Up Proof Fixed Point

- Output C is a fixed point – no more atoms could be added to C by further application of the rule of derivation.
- C is the least fixed point – there is no smaller fixed point.

Bottom-Up Proof Fixed Point

- Let I be the interpretation in which every atom in the least fixed point is true and every atom not in the least fixed point is false.
- I is a model of KB.
- Suppose I is not a model of KB.
- There is a clause $h \leftarrow a_1 \wedge \ldots \wedge a_m \in KB$ which is false in I.
- h is false and $a_1 \wedge \ldots \wedge a_m$ is true.
- $a_1, \ldots, a_m$ are in the fixed point, and h is not in the fixed point.
- The rule of derivation can be used to add h to the fixed point, a contradiction to it being the fixed point.
- Therefore, I is a model of KB.
- I is the minimal model: it has the fewest true propositions. Every other model must also assign the atoms in C to be true.

Bottom-Up Proof is Complete

- Suppose $KB \models g$.
- g is true in every model of KB.
- g is true in the minimal model.
- g is in C.
- $KB \vdash g$.

Bottom-Up Proof Complexity

- Each definite clause is used at most once.
- Number of times the loop is repeated is bounded by the number of definite clauses in KB.
- Time complexity is linear in the size of the knowledge base
- If KB is indexed by the definite clauses so that the inner loop is carried out in constant time.

End